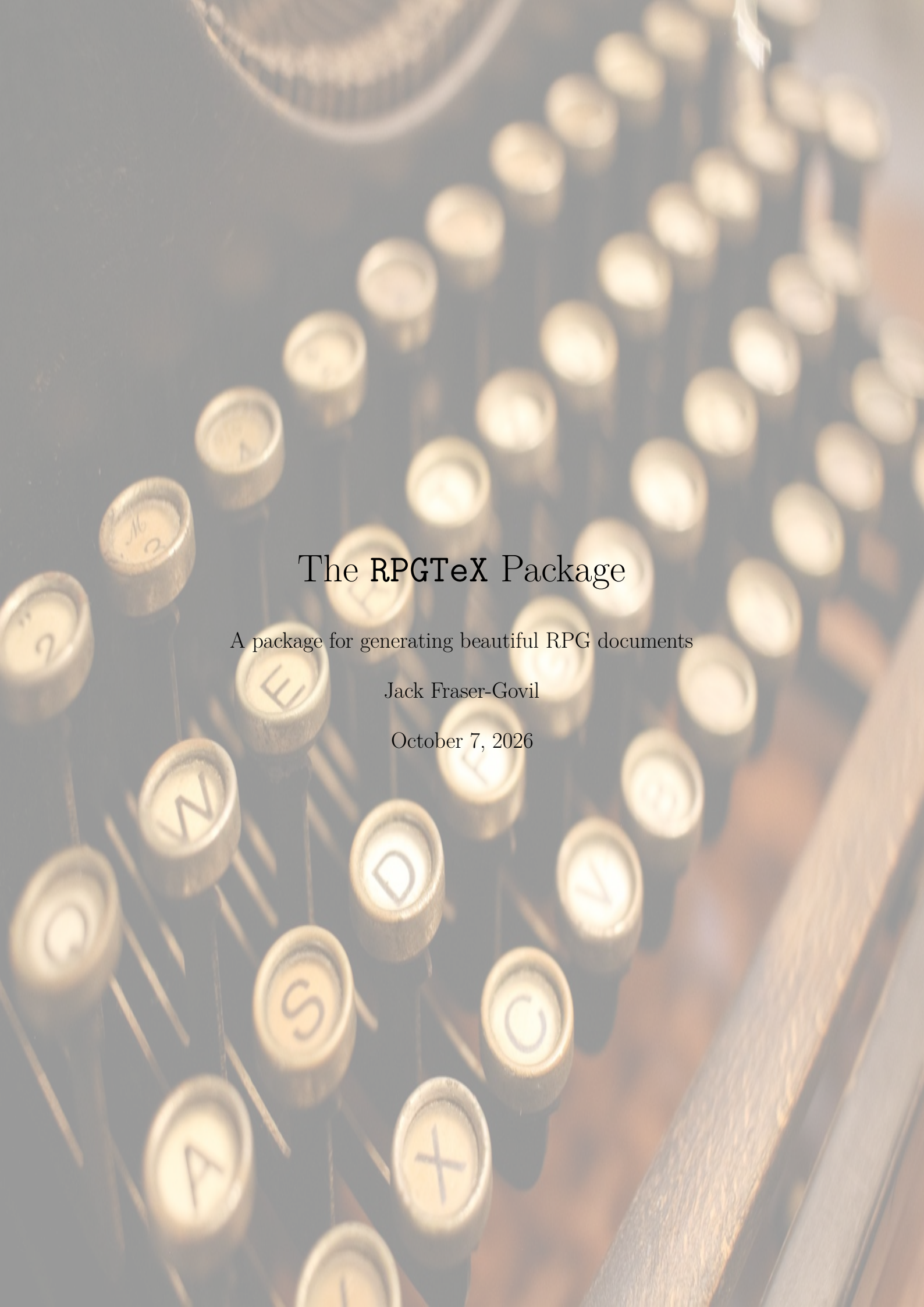




The **RPGTeX** Package

A package for generating beautiful RPG documents

Jack Fraser-Govil

October 7, 2026

Welcome to the *RPGTeX* package. This L^AT_EX package is designed to allow users to flexibly typeset documents associated with Role Playing Games such as *Dungeons & Dragons* – and many more besides. This packages defines a central engine: `rpgcore` which define a number of useful functions and classes, and a flexible set of `themes` which control how those commands are rendered in the final document.

Getting Started

This document is the full reference manual. For a quick example of how to get started with your first *RPGTeX* document, see the [example documents](#), which demonstrate the basic features of this package in a readable fashion.

Attribution & License

This package would not have been possible without the team who developed [its predecessor](#), the ‘*DND 5e LaTeX Template*’. That code was released under an MIT license, the text of which can be found in the `LICENSE` file. `rpgtex` is released under the same license.

AI Usage Declaration

The author used a handful of AI chat tools during development to help guide their thinking. The package itself was written entirely by hand: no AI tools were used to write any of its code. An AI assistant (Claude) helped edit this documentation. All mistakes and bad decisions are organic in nature.

Contents

1: Installing R ^P G ^T E ^X		
Getting R ^P G ^T E ^X	1	
Compiling R ^P G ^T E ^X Documents	2	
Part I: R^PG^TE^X For Users		
2: Package & Class Usage		
Choosing a Class.....	4	
Standalone Package	4	
rpgbook.....	5	
rpghandout.....	5	
rpgcard.....	5	
rpgdeck.....	5	
3: Options & Configuration		
Package Options	7	
Colors.....	9	
Lengths	10	
Command Line Interface.....	10	
4: Commands & Macros		
Themes & Colours	12	
Document Structure.....	12	
Text Utilities.....	15	
Full-Page Images.....	17	
Switches	17	
5: Environments		
Callouts: RpgNarration, Sidebar & Tip.....	19	
RpgTables	20	
RpgAbstracts	21	
RpgFiligree.....	21	
RpgSecrets	22	
RpgMaps.....	23	
RpgCards	24	
Rule Environments.....	25	
Part II: R^PG^TE^X For Designers		
6: Building a Theme		
Theme Files	28	
Conditional Code.....	29	
Key-Value Options.....	30	
7: Fonts		
Font Commands.....	33	
Font Elements	34	
8: Page Design		
Backgrounds & Footers.....	36	
Title & Part Pages.....	36	
9: Styling Elements		
Box Formatters	38	
Other Formatters	38	
10: Rule Environments		
Creating Rule Environments.....	42	
Properties.....	43	
Formats.....	43	
Part III: Appendices		
Appendix A: The rpgl ^a tex Compiler	47	
Appendix B: Dependencies	48	
Fonts	48	
Appendix C: Image Credits	49	
Appendix D: Index	50	

Chapter 1: Installing R^PG^TE^X

Getting R^PG^TE^X

R^PG^TE^X may be installed in several ways, described below. The package dependencies can be found in Appendix B (page 48). Once installed, R^PG^TE^X attempts to locate its internal resources (fonts and images) automatically. If this fails, manual configuration may be required, see page 2.

Operating Systems

The commands below are written assuming a Linux or macOS terminal. Windows users should consult the section on page 1 for more information.

Direct Installation

The simplest approach is to install R^PG^TE^X on the `texmf` path, where the compiler will find it automatically:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex" # make sure the directory exists
git clone https://github.com/DrFraserGovil/rpgtex.git "$(kpsewhich -var-value
    TEXMFHOME)/tex/latex/rpgtex"
```

This clones the repository into your personal L^AT_EX directory, which is typically `~/texmf` on Linux, `~/Library/texmf` on macOS, and `C:\Users\<username>\texmf` on Windows.

Indirect Installation

If you intend to modify R^PG^TE^X – for example, by creating a new theme – it is more convenient to keep it somewhere easily accessible. First, clone the repository into a location of your choice:

bash

```
git clone https://github.com/DrFraserGovil/rpgtex.git ~/path/to/rpgtex
```

The package must then be made visible to the compiler, in one of two ways.

Using TEXINPUTS

The `TEXINPUTS` environment variable adds directories to the compiler's search path. The `//` includes all subdirectories of the given directory, and the trailing colon keeps the standard search locations:

bash

```
export TEXINPUTS=~/path/to/rpgtex//:
```

The exact command depends on your shell; add it to your shell's configuration file to make it permanent.

Using a Symbolic Link

Alternatively, a symbolic link from the `texmf` directory to the repository allows the compiler to behave exactly as if R^PG^TE^X had been installed directly:

bash

```
mkdir -p "$(kpsewhich -var-value TEXMFHOME)/tex/latex"
ln -sf ~/path/to/rpgtex "$(kpsewhich -var-value TEXMFHOME)/tex/latex/rpgtex"
```

Windows

R^PG^TE^X works on Windows, with a few differences from the instructions above:

- Run the git commands using Git Bash for Windows
- Use the direct installation. Symbolic links and `TEXINPUTS` behave differently on Windows, and are best avoided.
- If you use MiKTeX, it does not automatically detect new files in its directories. After installing, register the folder containing **RPGTeX** as a root directory in MiKTeX Console (if it is not already one), and then refresh the file name database (*Tasks* → *Refresh file name database*). TeX Live requires no extra steps.
- Run the provided Python scripts through Python explicitly, for example `python rpglatex document.tex`. The `rpglatex` compiler does not yet open the finished PDF automatically on Windows.

Overleaf (Not Recommended)

We do not recommend Overleaf: the compilation time allowed by its free tier is too short for documents which use complex packages such as this one. Nevertheless, **RPGTeX** can be used there as follows:

1. Download the **RPGTeX** repository from GitHub as a ZIP archive.
2. On Overleaf, create a new project using *Upload Project*, and upload the ZIP archive.

Compiling RPGTeX Documents

RPGTeX uses the `fontspec` package to load its fonts, and so must be compiled with `xelatex` or `lualatex` in almost all cases. An exception is if **RPGTeX** is loaded as a package without the `font` option (page 8):

```
xelatex main.tex # works
lualatex main.tex # works
pdflatex main.tex # fails when the font option is active
```

Path Configuration

In most cases, no further steps are required to compile a document, since **RPGTeX** finds its own installation path, and stores the value in the following variable:

`\RpgPackagePath` (*Variable*)

The path to the root directory of the **RPGTeX** installation. **RPGTeX** uses it to load resources, such as fonts, which `TeX` cannot find relative to the package itself. It is set by the first of the following which is available:

1. **The `rpglatex` compiler** (page 47), which defines it before the document is read.
2. **A configuration file**, `core/rpg-config.cfg`, within the installation
3. **Automatically**. **RPGTeX** asks `kpsewhich` where `rpgtex.sty` is installed. This uses the compiler's restricted shell escape, which TeX Live enables by default, so no extra flags are needed.

If none of these succeeds, compilation stops with an error describing these options.

Failed Configuration

Most users will rely on the third option, the automatic configuration. However, this may fail if `LaTeX` is (for example) running in safe mode. In such cases the user must either use the `rpgtex` compiler, run the python script located at `scripts/configure-rpgtex`, or manually create the configuration file:

```
cd <rpgtex root directory>
echo '\edef\RpgPackagePath{"$(pwd)"}' > core/rpg-config.cfg
```

If **RPGTeX** is later moved, the file must be deleted or recreated.

A detailed pencil sketch of a person sitting at a desk in a room filled with plants and a large window. The person is seen from the side, looking down at their work. The desk is cluttered with various items, including a small box and some papers. To the right of the desk, there are several potted plants, including a large one in a dark pot. The background features a large window with a grid pattern, through which more plants are visible. The overall style is a fine-lined pencil drawing with a lot of texture and detail.

PART I
RPGTeX For Users

Chapter 2: Package & Class Usage

RPGTeX may be loaded through one of its four classes, which set up the whole document, or as a package within a document of any other class. Most documents should use a class. The simplest complete RPGTeX document is:

```
\documentclass{rpgbook}
\begin{document}
  \chapter{The Adventure Begins}
  Your text here.
\end{document}
```

Options are given in the usual way, such as `\documentclass[theme=dnd, columns=1]{rpgbook}` (page 7).

Choosing a Class

Each class is suited to a different kind of document:

Class	Purpose
<code>rpgbook</code> (page 5)	Long documents, such as rulebooks and adventures, which are divided into parts and chapters.
<code>rpgcard</code> (page 5)	One or a few cards, such as an item or spell, cropped to size for viewing on screen.
<code>rpgdeck</code> (page 5)	Many cards, arranged on pages for printing.
<code>rpghandout</code> (page 5)	Shorter documents, such as player handouts, which are divided into sections.

What the Classes Provide

Compared with loading the package, every class:

- Activates the `font`, `background` and `sections` modules (page 7). Every class except `rpgcard` also activates the `title` module.
- Sets the page size (A4, except in the screen mode of `rpgcard`) and margins, and passes the `size`, `columns`, `openright` and `oneside` options to the underlying class.
- Applies RPGTeX's overall text layout: hyphenation is restricted, text is left-aligned unless the `justified` option is given, lists are spaced more compactly, and the table of contents is set in two columns unless the `nomultitoc` option is given.

Font Sizes

The `size` option sets the base font size, which must be one of the sizes accepted by the underlying class:

Classes	Sizes
<code>rpgbook</code> , <code>rpghandout</code>	8pt, 9pt, 10pt, 11pt, 12pt, 14pt, 17pt and 20pt
<code>rpgcard</code> , <code>rpgdeck</code>	10pt, 11pt and 12pt

Standalone Package

The package may be loaded into a document of any class:

```
\documentclass{<any-class>}
\usepackage[<options>]{rpgtex}
```

This loads the core commands and environments, and the theme (the `default` theme, unless the `theme` option is used). However, the modules which change the fonts, page backgrounds, section headings and title page are inactive unless they are enabled with the `font`, `background`, `sections` and `title` options, and none of the layout changes listed above (page 4) are made. This allows RPGTeX to be added to an existing document without changing its overall appearance.

rpgbook

The **rpgbook** class is designed for long documents, such as rulebooks, sourcebooks and adventures, which need to be organised into parts and chapters. It is the class used to typeset this manual.

It is based on the **extbook** class from the **extsizes** package, which behaves almost identically to the standard **book** class, but accepts a wider range of font sizes (page 4). All of the usual **book** commands are therefore available, including `\part`, `\chapter`, `\frontmatter`, `\mainmatter`, `\backmatter` and `\appendix`.

By default, `\maketitle` produces a full cover page (page 37). The **openright** option starts each chapter on a right-hand page, and the **oneside** option lays the document out for single-sided printing.

The page geometry is:

Element	Size
Left and right margins	0.65in
Top margin	0.4in
Bottom margin (from the text to the bottom of the page)	0.75in
Footer separation (from the text to the footer)	0.3in
Gap between columns	0.25in

rpghandout

The **rpghandout** class is designed for shorter documents, where the structure of a book is unnecessary: printouts for players, or short adventure modules.

It is based on the **extarticle** class from the **extsizes** package, which behaves almost identically to the standard **article** class, but accepts a wider range of font sizes (page 4). The highest sectioning level is therefore `\section`.

By default, `\maketitle` produces a compact title at the top of the first page, rather than a cover page (page 37). It is often followed by a short summary, using the **abstract** environment (page 21).

The page geometry is the same as that of **rpgbook** (page 5).

rpgcard

The **rpgcard** class is designed for typesetting one or a few cards (page 25), primarily for viewing on screen, or for collecting into a deck later (page 5). The rule environments are displayed as cards: the class turns on the switch of every cardable rule environment (page 42) which exists when the class is loaded. This includes the standard environments, such as **RpgItem** and **RpgSpell**, and any created by the theme given in the class options, but not those created later (in the preamble, or by a theme loaded within the document), whose switches must be turned on with `\RpgSwitch` (page 18). Other switches, such as **ShowSecrets**, are unaffected.

The class has two modes, chosen by the **bg** option (page 7):

Screen mode (**bg=full**, the default): the class is based on the **standalone** class, and each page is cropped to fit its contents exactly, giving a file which contains just the card.

Print mode (**bg=print** or **bg=none**): the class is based on the **article** class, and the cards are placed on A4 pages, with a margin of 5pt on each side. For printing large numbers of cards, **rpgdeck** is more suitable.

Cards cannot have a title page or a table of contents, and `\maketitle` raises an error. The **columns** option is ignored, since cards are always laid out in a single column.

In screen mode, the whole document is typeset as a single line, so everything should be placed within a card or rule environment: loose text and headings outside of a card cause errors. Several cards are placed side by side.

rpgdeck

The **rpgdeck** class is designed to collect a large number of cards (page 25) into a single document for printing, including cards which were written as separate **rpgcard** documents.

It is based on the standard **article** class, on A4 pages with a margin of 5pt on each side, and the cards are placed side by side, filling each page in turn. As in **rpgcard**, the rule environments are displayed as cards: the class turns

on the switch of every cardable rule environment which exists when the class is loaded, including the standard ones and any created by the theme given in the class options. Environments created later (in the preamble, or by a theme loaded within the document) must have their switches turned on with `\RpgSwitch` (page 18). The `columns` option is ignored, and no page numbers or footers are shown. Unlike `rpgcard`, a deck may have a title: `\maketitle` produces a compact title at the top of the first page (page 37).

With the default `bg=full`, the theme's paper background is drawn behind the cards across the whole page. When printing, `bg=print` avoids this.

Including rpgcard Documents

The `rpgdeck` class loads the `standalone` package with the `subpreambles=true` option. This allows a complete `rpgcard` document, with its own `\documentclass` and `document` environment, to be included in a deck with `\input`. For example, given the card:

```
% spell1.tex
\documentclass[theme=dnd]{rpgcard}
\begin{document}
  \begin{RpgSpell}{Fireball}
    ...
  \end{RpgSpell}
\end{document}
```

which may be compiled on its own (and, for example, sent to a player), several such cards may be collected into a deck:

```
% deck.tex
\documentclass[theme=dnd, bg=print]{rpgdeck}
\begin{document}
  \input{spell1}
  \input{spell2}
  ...
\end{document}
```

The preambles of the included cards are collected and added to the preamble of the deck, so any definitions they make are shared between all of the cards. Two cards which define the same command therefore conflict, and every card uses the last definition. To avoid this, definitions which differ between cards should be made within their `document` environments.

The `\documentclass` line of an included card is ignored, along with its options. The cards are therefore typeset with the deck's theme and options, which may differ from the ones used when the card was compiled on its own; `\RpgSetTheme` (page 12) may be used within the deck to change the theme.

Chapter 3: Options & Configuration

This chapter describes the ways in which the behaviour of `RPGTeX` may be configured without writing any new code: the options which may be passed to the package or class, the named colours and lengths used by its elements, and the command line interface for changing these options at compile time.

Package Options

Whether `RPGTeX` is loaded directly as a package or indirectly through one of its classes, options may be passed to it which change the behaviour of the resulting document:

```
\documentclass[<options>]{rpgbook}
\usepackage[<options>]{rpgtex}
```

Each option takes one of two forms:

Key-value options: The name of the option is followed by an equals sign and a value:

```
\usepackage[bg = print, theme = dnd]{rpgtex}
```

Flags: Only the name of the option is provided. Its presence enables the associated behaviour:

```
\usepackage[justified]{rpgtex}
```

The available options are:

background <flag>

If present, activates the ‘paper background’ modules, which control the paper colour and background images, the footer decorations, and the spacing around floats.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

bg = full / print / none

Controls the behaviour of the background and footer decorations. The value must be one of the three listed above, else an error is thrown. The default value is `full`.

The most visible effect of this option is to control whether the ‘background paper’ set by `\RpgThemePaperImage` (page 36) and `\RpgThemePaperColor` (page 36), and the footer decorations set by `\RpgThemeFooterDecoration` (page 36), appear in the document:

Value	Paper	Footer Decorations
full	✓	✓
print	✗	✓
none	✗	✗

This option also changes the behaviour of the `rpgcard` class (page 5), and other environments may similarly alter their colours or layouts in response to its value.

columns = 1 / 2

Places the document into either `onecolumn` or `twocolumn` mode. The default value is 2.

More advanced column layouts require the user to manually use the `multicols` environment.

This option only has an effect when using one of the `RPGTeX` classes.

`font <flag>`

If present, activates the font modules, applying the package fonts wherever possible.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

`justified <flag>`

If present, sets the text to be justified throughout the document. By default, text is left-aligned ('ragged right').

`noemph <flag>`

If present, the default behaviour of `\emph` is left unchanged. By default, `\emph` is redefined to be an alias of `\RpgKey` (page 16).

`nomultitoc <flag>`

If present, disables the multi-column table of contents.

`oneside <flag>`

If present, places the document into one-sided mode. This disables alternating footers, but allows for consistent asymmetric margins (as in this document, where the left margin is enlarged to accommodate macro names).

This option only has an effect when using one of the `RPGTeX` classes.

`openright <flag>`

If present, places the document into right-opening mode, such that chapters always begin on a right-hand page. By default, chapters may begin on any page (see the documentation of the `book` class for more details).

This option only has an effect when using one of the `RPGTeX` classes.

`sections <flag>`

If present, activates the sections modules, including the part-page overloads and any theme behaviour which modifies the section commands.

This flag is set by default when loading a class, but must be passed explicitly to enable these modules when loading the package directly.

`size = <font-size>pt`

Sets the base font size of the document, if permitted by the parent class. The default value is 10pt.

This option only has an effect when using one of the `RPGTeX` classes.

The allowed values depend on the class being used (page 4).

`theme = <theme name>`

The initial value passed to `\RpgSetTheme` (page 12) once package initialisation is complete.

The default value is `default`, which activates the Default Theme (page ??).


```
themepath = </path/to/themes/>
```

Calls `\RpgSetThemePath` (page 12) before the theme is loaded, allowing custom themes to be used.

The default value is `\RpgPackagePath/themes`, which `TEX` locates within the `RPGTeX` installation. A theme called `name` is then expected to be stored in `<themepath>/name/name.cfg`.

```
title <flag>
```

If present, activates the titling module and any theme presets which alter the behaviour of `\maketitle`.

This flag is set by default when loading a class, but must be passed explicitly to enable this module when loading the package directly.

Colors

`RPGTeX` defines a number of named colors which are used by its various elements. The default values listed below may be overridden by the active theme.

A Note on Spelling

Although this document otherwise uses British spelling, this section follows the American spelling of ‘color’ used throughout `LATEX` and `xcolor`. The color names below must be written exactly as given, including their spelling and capitalization.

An exception is in Gray/Grey, as I personally cannot remember which is the ‘right’ way to spell it, so I have aliased most colours to use both spellings.

Co-varying Colors

The following colors are coupled together, and by default all take the same value:

Color	Usage	Default
<code>SidebarColor</code>	The background color of the <code>RpgSidebar</code> (page 19) environment.	<code>ThemeColor</code>
<code>TableColor</code>	The background color of alternate rows in an <code>RpgTable</code> (page 20).	<code>ThemeColor</code>
<code>ThemeColor</code>	A general-purpose color from which the other co-varying colors inherit.	<code>white!60!black</code>
<code>TipColor</code>	The background color of the <code>RpgTip</code> (page 19) environment.	<code>ThemeColor</code>

Calling `\RpgSetThemeColor` (page 12) assigns a new value to `ThemeColor` and all three co-varying colors simultaneously:

```
\RpgSetThemeColor{blue!20!white}
```

Independent Colors

The remaining colors are set individually:

Color	Usage	Default
<code>ContourInnerColor</code>	The default color of the inner text produced by <code>\RpgContour</code> (page 15).	<code>black</code>
<code>ContourOuterColor</code>	The default color of the outer contour drawn around text by <code>\RpgContour</code> (page 15).	<code>white!50!black</code>
<code>FiligreeBackColor</code>	The background color of the <code>RpgFiligree</code> (page 21) environment.	<code>white</code>
<code>FiligreeColor</code>	The color used to draw the filigree frame applied by the <code>rpgfiligree</code> style.	<code>black</code>
<code>NarrationColor</code>	The background color of the <code>RpgNarration</code> (page 19) environment.	<code>F7F2E5</code> (HTML)
<code>PaperColor</code>	The background color of each page, applied only when the paper background is enabled (see the <code>bg</code> option, page 7).	<code>white</code>
<code>SecretColor</code>	The default frame color of the <code>RpgSecret</code> (page 22) environment.	<code>black!30!white</code>

Any of these colors, including the co-varying colors, may be modified individually using the standard `xcolor` interface:

```
\definecolor{NarrationColor}{HTML}{FFFFFF}
\colorlet{SidebarColor}{red!10!white}
```

Modifying a co-varying color in this way changes only that color, leaving the others unaffected.

Lengths

A small number of lengths used by **RPGTeX** may be changed directly, without any special interface. As with the colors, the active theme may override their default values.

`\RpgDropCapSeparation` (Variable)

The horizontal space between the enlarged letter created by `\RpgDropCap` (page 16) and the text which follows it. It is a standard $\text{\LaTeX}$ length, and is changed with `\setlength`:

```
\setlength{\RpgDropCapSeparation}{0.5em}
```

The default value is the width of a single space.

`\RpgFiligreeSize` (Variable)

The size of the decorative elements of the filigree frame (page 21), which the theme uses to scale its design. Unlike a standard length, it is stored as a macro, and so is changed with `\renewcommand`:

```
\renewcommand{\RpgFiligreeSize}{8pt}
```

The default value is 5pt.

Command Line Interface

$\text{\LaTeX}$ does not natively provide a ‘command line interface’ through which a document may be modified at compile time: any change must be made within the source file itself. It is, however, often convenient to compile the same document in several configurations without editing it – most notably to produce both a standard version and a printer-friendly version (using `\bg = print`, which removes the paper background to reduce ink usage).

To support this, **RPGTeX** checks for a pseudo command-line variable when processing its options. Because $\text{\TeX}$ treats the text passed to it on the command line as the beginning of its input stream, this variable can be defined before the document itself is read.

`\RpgCMD` (Variable)

If defined, holds a comma-separated list of package options (page 7), which are applied after all other options have been processed. The options are applied in the following order, with later values overriding earlier ones:

1. The defaults set by the class (if one is being used).
2. The options passed in the document, via `\documentclass` or `\usepackage`.
3. The contents of `\RpgCMD`.

Since the options are processed before they are passed to the parent class, options which affect the class (such as `columns` or `size`) may also be set in this way.

`\RpgCMD` is defined by passing a short piece of $\text{\TeX}$ code to the compiler before the document is input:

```
xelatex "\def\RpgCMD{<rpg-options>} \input{<document>}"
```

This compiles `<document>` as though `<rpg-options>` had been passed to the class or package. For example, the following produces a printer-friendly, single-column version of a document without modifying the source:

```
xelatex "\def\RpgCMD{bg=print, columns=1} \input{document.tex}"
```

The `rpglatex` compiler (page 47) performs this insertion automatically: the `-p` (`--print`) flag sets `bg=print`, allowing the user to switch between the standard and print versions of a document with a single flag:

```
rpglatex document.tex -p
```

Chapter 4: Commands & Macros

Themes & Colours

```
\RpgSetTheme { name }
```

Loads the theme called `name`, replacing the appearance of any previously loaded theme. The theme is first searched for at `<themepath>/<name>/<name>.cfg`; if no such file exists, `name` is instead treated as a direct path to a theme file. An error is raised if neither file exists.

The initial theme is set by the `theme` option (page 7). `\RpgSetTheme` may be called at any point in the document, but since its effects are immediate, it is best called at the start of a new page.

```
\RpgSetThemeColor { colour }
```

Sets `ThemeColor` and all of the co-varying colours to `colour` (page 9).

```
\RpgSetThemePath { path }
```

Sets the directory searched by `\RpgSetTheme`, allowing custom themes to be stored outside of the R_PG_TE_X installation. The path is not checked until a theme is loaded. The initial value is set by the `themepath` option (page 9).

Document Structure

R_PG_TE_X extends the standard L^AT_EX commands for structuring a document. It provides title and part pages designed by the active theme, records the names of the current headings so that they may be reused elsewhere, and allows ‘fake’ headings to be added to the table of contents.

The Title & Sections Options

Much of the behaviour described in this section is provided by redefining standard L^AT_EX commands, and is controlled by two package options (page 7). Both are active by default when using one of the R_PG_TE_X classes; otherwise, they must be passed to the package explicitly.

title Redefines `\maketitle` to produce the title designed by the active theme (equivalent to calling `\RpgMakeTitle`). Otherwise, `\maketitle` retains its original L^AT_EX behaviour.

sections Activates *sections mode*, which:

- Redefines `\chaptermark`, `\sectionmark`, `\subsectionmark` and `\subsubsectionmark` so that, in addition to their usual behaviour, they store the name they are given (page 14).
- Sets `secnumdepth` to 0, so that sections and their subdivisions are unnumbered.
- Replaces `\part` with a version which records the part name and draws a full-page part page using the active theme. The starred form, `\part*`, records the name but otherwise falls back to the original L^AT_EX command.

Themes may additionally alter the formatting of the sectioning commands when sections mode is active.

All of the other commands in this section, including `\RpgMakeTitle`, are available regardless of these options, although the stored heading names are only updated in sections mode.

Title Pages

The title page draws on the usual `\title` and `\author` commands, along with the following additions.

```
\cover { path/to/image }
```

Sets an image to be used as the full-page background of the cover page created by `\maketitle` in *cover* mode (see below). The theme’s title design is drawn over the top of the image.

The default value is empty, in which case the title design is drawn on a plain page.

`\maketitle { }`

\RpgMakeTitle Creates the title of the document using the design provided by the theme (only when the `title` option is active; see above). There are two title modes:

cover A full-page cover, drawn over the `\cover` image (if one has been set). This is the default for the `rpgbook` class.

header A title heading placed at the top of the first page of text.

The mode is selected with `\RpgThemeTitleMode{ cover / header }`. If the theme does not define a title for the active mode, the original L^AT_EX `\maketitle` is used instead.

`\RpgMakeTitle` is equivalent to `\maketitle`, but remains available even if the `title` option is not active, or if another package redefines `\maketitle`.

`\RpgCoverTransparency { value }`

Fades the `\cover` image towards white, which can help to keep the title legible over a busy image. The `value` is the opacity of a white layer placed over the image, between 0 (the image is unaltered) and 1 (the image is completely hidden). The default value is 0.

`\RpgGetAuthor (Variable)`

\RpgGetAuthor* The author set by `\author`, formatted with the `\RpgFontSubTitle` (page 34) font element. The starred form gives the author as plain text.

`\RpgGetSubtitle (Variable)`

\RpgGetSubtitle* The subtitle set by `\subtitle`, formatted with the `\RpgFontSubTitle` (page 34) font element. The starred form gives the subtitle as plain text.

`\RpgGetTitle (Variable)`

\RpgGetTitle* The title set by `\title`, formatted with the `\RpgFontTitle` (page 34) font element (page 33). The starred form gives the title as plain text, without applying the font.

`\subtitle { text }`

Sets the subtitle of the document. As with `\title` and `\author`, this has no visible effect by itself: the subtitle is displayed by `\maketitle` if the theme includes it. The default value is empty.

Part Pages

`\part (*)[<image>]{ name }`

In sections mode (page 12), `\part` is redefined to create a full-page part page using the design provided by the theme. Its behaviour depends on the form used:

\part{name} Adds the part to the table of contents and creates a part page on a plain background, or over the default part image if one has been set with `\RpgDefaultPartImage`.

\part[image]{name} As above, but uses `image` as the full-page background.

\part*{name} Uses the original `\part` command of the underlying class. Any image argument is ignored.

In all cases, `name` is recorded in `\RpgPartName` (page 14).

`\RpgDefaultPartImage { path/to/image }`

Sets an image to be used as the full-page background of every part page which is not given its own image. By default, no image is used.

`\RpgPartTransparency { value }`

Fades all subsequent images set by `\part` image towards white, which can help to keep the text legible over a busy image. The **value** is the opacity of a white layer placed over the image, between 0 (the image is unaltered) and 1 (the image is completely hidden). The default value is 0.

Section Names

Many RPG documents display the name of the current chapter or section outside of the heading itself – in a running header, a footer, or a decorative page border, for example. To support this, **RPGTeX** records the names of the current part, chapter, section, subsection and subsubsection, so that they may be reused anywhere in the document.

These names are only recorded in sections mode (page 12).

Each of the following variables is initially blank, and holds its value until the next command of the same level is called: beginning a new chapter does not, for example, reset `\RpgSectionName`.

`\RpgChapterName` (*Variable*)

The name given to the last `\chapter` command, or set by calling `\chaptermark`. Starred chapters (`\chapter*`) do not update this value.

`\RpgPartName` (*Variable*)

The name given to the last `\part` or `\part*` command.

`\RpgSectionName` (*Variable*)

The name given to the last `\section` command, or set by calling `\sectionmark`. Starred sections (`\section*`) do not update this value.

`\RpgSubsectionName` (*Variable*)

The name given to the last `\subsection` command, or set by calling `\subsectionmark`. Starred subsections (`\subsection*`) do not update this value.

`\RpgSubsubsectionName` (*Variable*)

The name given to the last `\subsubsection` command, or set by calling `\subsubsectionmark`. Starred subsubsections (`\subsubsection*`) do not update this value.

`\RpgGetName { part / chapter / section / subsection / subsubsection }`

Returns the name stored for the given level (equivalent to the corresponding variable above). Any other input results in an error.

Since these names (other than that of the part) are taken from the **mark** commands, they may be set manually, without creating a new heading, by calling the relevant **mark** command directly.

```
We are in Chapter
'\RpgGetName{chapter}', section
'\RpgGetName{section}'.

\subsubsection{An example subsubsection}

Now I am in
'\RpgGetName{subsubsection}'.
```

We are in Chapter 'Commands & Macros',
section 'Document Structure'.

An example subsubsection

Now I am in 'An example subsubsection'.

Fake Sections

It can be helpful to be able to integrate common RPG elements (such as full page maps or images) into the document structure. The following command provides a simple interface for spoofing in a section (or part, chapter etc.) into the table of contents, without actually invoking a sectioning command:

```
\RpgFake { level }{ name }
```

Mimics the creation of a new heading at the given `level` without printing a heading in the document. Specifically, it:

- Adds an entry called `name` to the table of contents at the given level, with a hyperlink anchor at the current position.
- Updates the stored name for that level, as if a real heading had been created. For `chapter` to `subsubsection` this is done by calling the corresponding `mark` command, so any running headers are also updated.

The `level` must be one of `part`, `chapter`, `section`, `subsection` or `subsubsection`. Any other value results in an error.

```
\subsubsection{Real Subsection}
I believe I am in
  '\RpgGetName{subsubsection}'.

\RpgFake{subsubsection}{Fake Subsection}
Now I believe I am in
  '\RpgGetName{subsubsection}'
```

Real Subsection

I believe I am in 'Real Subsection'.
Now I believe I am in 'Fake Subsection'

Text Utilities

RPGTeX provides a number of commands for applying decorative styles to text and transforming it in ways useful for writing in an RPG context.

Any fonts or styles they apply are those assigned by the theme (page 33).

```
\RpgContour [<options>]{ text }
```

Typesets `text` with a coloured outline (contour), using the `contour` package. The following options are available:

Option	Effect	Default
<code>inner</code>	The colour of the text itself.	<code>ContourInnerColor</code> (page 9)
<code>outer</code>	The colour of the outline.	<code>ContourOuterColor</code>
<code>style</code>	Formatting commands (such as <code>\bfseries</code> or <code>\Huge</code>) applied to the text.	(empty)

Contours cannot automatically linebreak, instead the text is split into lines at each `\\`, and each word is contoured individually so that spacing is preserved. As a consequence, formatting commands placed within `text` may only affect the first word: all formatting should therefore be passed through the `style` key.

```
\RpgContour[inner=red,
outer=black]{Contoured text}

\RpgContour[style=\Large\bfseries]{Styled
text \\ over two lines}
```

Contoured text
Styled text
over two lines

```
\RpgDice { expression }
```

Parses a dice expression of the form `<count>d<sides><modifier>` and typesets it in the format defined by the theme. The expression must satisfy the following rules:

- It contains a dice symbol (`d` or `D`), immediately followed by the number of sides.
- The dice symbol may be preceded by the number of dice. If it is omitted, the theme determines how the expression is displayed.
- Anything following the number of sides is treated as the modifier, which may be any numerical expression (such as `+3*2`). It is evaluated to a single number before being displayed.

Leading and trailing spaces are ignored. If the expression does not match this form, it is printed unchanged.

<code>\RpgDice{1d6-2}</code>	1d6-2
<code>\RpgDice{2D6 + 3*2^2}</code>	2d6+12
<code>\RpgDice{d8}</code>	d8
<code>\RpgDice{not a dice roll}</code>	not a dice roll

`\RpgDropCap` [`<options>`]{ `letter` }{ `text` }

Begins a paragraph with a large decorative initial (a ‘drop cap’), using the `lettrine` package. The `letter` is typeset in the `\RpgFontDropCap` (page 35) font element, and `text` – intended to be the remainder of the first line – in the `\RpgFontDropCapAfter` (page 35) font element.

The drop cap spans a number of lines set by the theme (four, by default), and is separated from the following text by the length `\RpgDropCapSeparation` (page 10). The optional argument accepts any `lettrine` options, which override these defaults.

<code>\RpgDropCap{T}{he example: this text runs over the first line, and then revert back to the normal font. It almost works! However, because it's wrapped in a text box, it goes slightly over the edges.}</code>	<i>T</i> <i>he example: this text runs over the first line, and then revert back to the normal font. It almost works! However, because it's wrapped in a text box, it goes slightly over the edges.</i>
--	---

`\RpgKey` { `text` }

`\emph` Typesets `text` in the `\RpgFontEmphasis` (page 34) font element, giving a consistent appearance to keywords (such as conditions or game terms).

When the `font` option is active, and unless the `noemph` option is passed (page 7), `\emph` is redefined to be an alias of `\RpgKey`. This replaces its usual ‘context-aware’ behaviour (italics within upright text, and vice versa) with the keyword style.

<code>\emph{text}</code>	<i>text</i>
<code>\RpgKey{more text}</code>	<i>more text</i>

`\RpgOrdinal` [`<format>`]{ `number` }

Typesets an integer followed by its English ordinal suffix (`st`, `nd`, `rd` or `th`), correctly handling 11th, 12th and 13th. The optional `format` is a command applied to the suffix only, such as `\textbf`.

<code>\RpgOrdinal{1}, \RpgOrdinal{12}, \RpgOrdinal{23}</code>	1st, 12th, 23rd
<code>\RpgOrdinal[\textbf]{133}</code>	133rd

`\RpgPage` [`<style>`]{ `label` }

Typesets ‘page’ followed by the page number of `label`, optionally enclosed in brackets. The `style` may be one of `t` (no brackets; the default), `p` (parentheses), `b` (square brackets) or `c` (curly braces). Any other value encloses the reference in question marks, to make the error easy to spot.

<code>\RpgPage{S:PackageOptions}</code>	
<code>\RpgPage[p]{S:PackageOptions}</code>	page 7
<code>\RpgPage[b]{S:PackageOptions}</code>	(page 7)
<code>\RpgPage[c]{S:PackageOptions}</code>	[page 7]
	{page 7}

`\RpgPlural (*)[<plural>]{ count }{ word }`

Typesets `count` followed by `word`, adding an ‘s’ to the word unless `count` is 1. Irregular plurals may be given in the optional argument, which replaces the word whenever `count` is not 1. The starred form omits the count, printing only the word.

<code>\RpgPlural{1}{hat}, \RpgPlural{2}{hat}</code>	1 hat, 2 hats
<code>\RpgPlural[octopodes]{1}{octopus},</code> <code>\RpgPlural[octopodes]{3}{octopus}</code>	1 octopus, 3 octopodes
<code>\RpgPlural*{3}{hat}</code>	hats

`\RpgSuperOrdinal { number }`

Equivalent to `\RpgOrdinal[\textsuperscript]{ number }` (page 16), automatically applying a superscript style command.

<code>\RpgSuperOrdinal{42}</code>	42 nd
-----------------------------------	------------------

Full-Page Images

We use the `eso-pic` library for formatting whole-page images and placing content at specific positions on the page.

`\RpgPut [<width>][<format>](<anchor>){ x }{ y }{ content }`

Places `content` at a fixed position on the page, measured from the bottom-left corner. It may only be used within a page overlay, such as the first argument of `\RpgWholePageImage`.

The content is set in a paragraph of the given `width` (default: `\linewidth`), with `format` applied at the start (default: `\centering`).

The `anchor` determines which point of the content is placed at (x, y): `c` (centre; the default), `t`, `b`, `l`, `r`, or a combination such as `tl`.

`\RpgWholePageImage [<overlay>]{ path/to/image }[<fade>]`

Places an image so that it fills the whole of a new page, clearing the page on either side. This is useful for maps, handouts and illustrations.

The optional `overlay` is drawn on top of the image, and may contain any number of `\RpgPut` commands. The final optional argument (given in square brackets, after the path) fades the image towards white, as described for `\RpgCoverTransparency` (page 12).

Switches

Switches are named true/false values which can be used to change the content of a document without editing its text. For example, `RPGTeX` uses switches to decide whether secrets are shown, and whether rule environments are displayed as cards. All switches are global: changing a switch affects the whole of the remaining document.

Switches are designed to act as boolean counterparts to traditional `TEX` counters, and so have a similar interface.

The primary use-case for switches in `RPGTeX` is for the ‘Cardable Environments’ , allowing the same document to typeset as both inline text, and as a ‘playing card module’ depending on the value of the associated switch.

```
\RpgIfSwitch { name }{ if-on }{ if-off }
```

Typesets `if-on` if the switch is on, and `if-off` otherwise.

```
\RpgNewSwitch { name }[<value>]
```

Creates a new switch called `name`, with an initial `value` (default: `false`). An error is raised if the switch already exists.

```
\RpgSwitch { name }{ value }
```

Sets an existing switch to the chosen value. Accepted input strings are:

Boolean State	Input Strings
True	on, On, true, True, 1
False	off, Off, false, False, 0

Any other value raises an error, as does a switch which does not exist.

```
\RpgToggle { name }
```

Turns an existing switch on if it is off, and off if it is on.

```
\RpgNewSwitch{HardMode}
```

The dragon has

```
\RpgIfSwitch{HardMode}{500}{200} HP.
```

```
\RpgToggle{HardMode}
```

The dragon has

```
\RpgIfSwitch{HardMode}{500}{200} HP.
```

The dragon has 200 HP.

The dragon has 500 HP.

Chapter 5: Environments

RPGTeX provides a number of environments for the elements which appear throughout RPG books: boxed text, tables, map keys, secrets and playing cards. Their appearance (fonts, colours and decoration) is set by the active theme, but may be adjusted through their options.

Many of these environments are built on the `tcolorbox` package. Unless stated otherwise, any option which is not recognised by an environment is passed on to `tcolorbox`, so all of its usual options (such as `colframe` or `breakable`) may also be used.

Callouts: RpgNarration, Sidebar & Tip

Callouts are boxed blocks of text which stand apart from the main text, the precise appearance determined by the theme. There are three types of callout, intended for different purposes:

RpgNarration	Text to be read aloud, such as the description of a scene at the start of an adventure.
RpgSidebar	Important information, such as key rules or summaries, which readers <i>should</i> pay attention to.
RpgTip	Helpful additions, such as tips and trivia, which are not essential but are too large for a footnote.

Each callout accepts the following options, in addition to the usual `tcolorbox` options:

Option	Effect	Default
color	The background colour of the box and its title, setting both <code>colback</code> and <code>colbacktitle</code> . An explicit <code>colback</code> or <code>colbacktitle</code> option takes priority over this value.	<code>NarrationColor</code> , <code>SidebarColor</code> or <code>TipColor</code> (page 9)
filigree	If present, decorates the box with the theme's filigree frame (page 21).	off

```
\begin{ RpgNarration } [<options>]
```

A box for text to be read aloud. It has no title, and its text is typeset in the `\RpgFontNarration` (page 35) font element on a `NarrationColor` background.

```
\begin{RpgNarration}[color=blue!30!white]
  This is text that you would read out
  loud to players, describing a
  scene. It will always be blue,
  even if the theme says otherwise
  -- because the optional argument
  takes priority.
\end{RpgNarration}
```

This is text that you would read out loud to players, describing a scene. It will always be blue, even if the theme says otherwise – because the optional argument takes priority.

```
\begin{ RpgSidebar } [<options>]{ title }
```

A box for important information, with a title. The title and body are typeset in the `\RpgFontSidebarTitle` (page 35) and `\RpgFontSidebarBody` (page 35) font elements, and the background is `SidebarColor`.

```
\begin{RpgSidebar}{A Sidebar}
  This is an important block of text,
  that you should pay attention to.
\end{RpgSidebar}
```

A Sidebar

This is an important block of text, that you should pay attention to.

```
\begin{ RpgTip } [<options>]{ title }
```

A box for helpful additions, with a title. The title and body are typeset in the `\RpgFontTipTitle` (page 35) and `\RpgFontTipBody` (page 35) font elements, and the background is `TipColor`.

```
\begin{RpgTip}{A Tip}
  This is some helpful - but not vital
  - text.
\end{RpgTip}
```

A Tip

This is some helpful - but not vital - text.

RpgTables

`\RpgRowStyle { format }`

Applies `format` to every cell of the current row of an `RpgTable`. It must be placed at the start of the row. This is the mechanism used to style the header row, and may be used to highlight any other row in the same way.

`\begin{ RpgTable } [<options>]{ columns }`

Typesets a table in the style of the theme. By default this is a `tabularx` table which fills the width of the line, so the usual column types (`l`, `c`, `r`, `p{<width>}`, ...) may be used alongside the stretching `X` column. `RPGTeX` also provides a centred stretching column, `Y`.

By default, the table:

- Typesets its first row as a header, in the `\RpgFontTableHeader` (page 35) font element, and the remaining rows in `\RpgFontTableBody` (page 35).
- Colours alternate rows in `TableColor` (page 9), beginning with the second row.

The following options are available:

Option	Effect	Default
<code>breakable</code>	If present, uses <code>xltabular</code> , allowing the table to break across pages. This is only possible in one-column mode.	off
<code>color</code>	The colour used for the coloured rows.	<code>TableColor</code> (page 9)
<code>color-both</code>	If present, colours every row.	off
<code>color-odd</code>	If present, colours the odd rows (including the header) instead of the even rows.	off
<code>filigree</code>	If present, wraps the table in an <code>RpgFiligree</code> (page 21).	off
<code>filigree-hgap</code>	The horizontal padding between the filigree and the table.	1pt
<code>filigree-opacity</code>	The opacity of the filigree's background.	1
<code>headskip</code>	The vertical space below the title.	5pt plus 2pt minus 2pt
<code>intro</code>	Introductory text, typeset between the title and the table.	(empty)
<code>introskip</code>	The vertical space below the introduction.	5pt plus 2pt minus 2pt
<code>noheader</code>	If present, the first row is typeset in the same style as the rest of the table.	off
<code>simple</code>	If present, uses a plain <code>tabular</code> , whose width is determined by its contents. The <code>width</code> , <code>breakable</code> and <code>noheader</code> options are then ignored, and the first row is not styled as a header.	off
<code>title</code>	A title, typeset above the table in the <code>\RpgFontTableTitle</code> (page 35) font element.	(empty)
<code>vskip</code>	The vertical space above and below the table.	9pt plus 3pt minus 3pt
<code>width</code>	The width of the table.	<code>\linewidth</code>

```

\begin{RpgTable}[width=0.85\linewidth,
color=green!30!white]{lX}
  Header 1 & Header 2
  \\
  Text & Text fills up to 85\% of the
    line width then breaks
  \\
  Alternating & This row is transparent
  \\
  Colour & but this one is green
\end{RpgTable}

%suppress header formatting, add title
and filigree:
\begin{RpgTable}[title={Test
Table},noheader, filigree]{XlX}
  Plain & Header & Text
  \\
  Now there's no & difference &
    between the header and the main
  \\
  body
\end{RpgTable}

```

Header 1	Header 2
Text	Text fills up to 85% of the line width then breaks
Alternating	This row is transparent
Colour	but this one is green

Test Table				
Plain		Header	Text	
Now	there's	difference	between	the
no			header	and
			the main	
body				

RpgAbstracts

```
\begin{ RpgAbstract } [<title>][<width>][<gap>]
```

abstract Typesets a short, centred summary block, such as the introduction to an adventure. The title (default: ‘Abstract’) is typeset in the `\RpgFontAbstractTitle` (page 35) font element, and the body in `\RpgFontAbstractBody` (page 35). The block has the given width (default: `0.7\linewidth`), with a vertical gap above and below it (default: 15pt). In two-column mode, the block spans both columns. The standard `abstract` environment is redefined to be identical to `RpgAbstract`.

```
\begin{ summary } [<title>][<width>][<gap>]
```

Identical to `RpgAbstract`, but with the default title ‘Summary’.

RpgFiligree

Filigree is a decorative frame, drawn around the outside of a box in place of its usual border. Its design is provided by the theme, and it is drawn in `FiligreeColor` (page 9).

```
\begin{ RpgFiligree } [<options>]
```

A plain box decorated with the filigree frame, with a `FiligreeBackColor` background. It is equivalent to an untitled `RpgTip` (page 19) with the `filigree` option, and accepts the same options.

The filigree may also be added to any `tcolorbox` by including the `rpgfiligree` style in its options, or to a callout or table with the `filigree` option.


```

%% As environment
\begin{RpgFiligree}
  A basic filigree-wrapped tbox.
\end{RpgFiligree}

%% As option
\begin{RpgSidebar}[filigree]{Fancy
Sidebar}
  A very fancy sidebar
\end{RpgSidebar}

```

A basic filigree-wrapped tbox.

Fancy Sidebar

A very fancy sidebar

RpgSecrets

Game Masters often need notes which their players should not see. The `RpgSecret` environment allows the same document to serve both purposes, by marking text as secret and choosing whether to reveal it.

```
\begin{ RpgSecret } [<options>]{<public-text>}
```

Marks the body of the environment as secret. Its appearance depends on the `ShowSecrets` switch (page 17), which is off by default:

Switch	Display
Off	Only the (optional) <code>public-text</code> is shown, as ordinary text, with nothing to suggest that anything has been withheld.
On	The secret text is shown in a highlighted box, labelled as hidden. If <code>public-text</code> was given, it is shown in the lower part of the box, labelled as the publicly available version.

Secrets are revealed for the remainder of the document with `\RpgSwitch{ ShowSecrets }{ on }`. The following options are available:

Option	Effect	Default
color	The colour of the box's frame.	<code>SecretColor</code>
label-public	The label above the public text.	'Normal text'
label-secret	The label above the secret text.	'Hidden text'
switch	The switch which controls this secret. Using a different switch allows separate groups of secrets to be revealed independently.	<code>ShowSecrets</code>

The labels are typeset in the `\RpgFontSecretTitle` (page 35) font element.

RpgSecret: Hidden

```

\RpgSwitch{ShowSecrets}{false}

\lorumIpsumOne%Snippets of standard lorem ipsum
\begin{RpgSecret}{}
  The players don't know it yet, but all this
  pseudo-latin text is a hint that devils
  are involved!
\end{RpgSecret}
\lorumIpsumTwo{}
\begin{RpgSecret}{
  Logos kykloforia, thelxis kai ethos,
  tyrannis kai epilysis. Aei theria plaka,
  kyrios meion egestas, anagkas proin eira
  kai theria.
}

  This bit of text is noteworthy - it's been
  added in after the fact
\end{RpgSecret}
\lorumIpsumThree

```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Logos kykloforia, thelxis kai ethos, tyrannis kai epilysis. Aei theria plaka, kyrios meion egestas, anagkas proin eira kai theria. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc.

```
%exactly the same as above, but reveal the secrets!
\RpgSwitch{ShowSecrets}{true}

\lorumIpsumOne%Snippets of standard lorem ipsum
\begin{RpgSecret}{}
  The players don't know it yet, but all this
  pseudo-latin text is a hint that devils
  are involved!
\end{RpgSecret}
\lorumIpsumTwo{
\begin{RpgSecret}{
  Logos kykloforia, thelxis kai ethos,
  tyrannis kai epilyssis. Aei theria plaka,
  kyrios meion egestas, anagkas proin eira
  kai theria.
}

  This bit of text is noteworthy - it's been
  added in after the fact
\end{RpgSecret}
\lorumIpsumThree
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

Hidden text

The players don't know it yet, but all this pseudo-latin text is a hint that devils are involved!

Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

Hidden text

This bit of text is noteworthy - it's been added in after the fact

Normal text

Logos kykloforia, thelxis kai ethos, tyrannis kai epilyssis. Aei theria plaka, kyrios meion egestas, anagkas proin eira kai theria.

Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc.

RpgMaps

Adventures often describe a location as a numbered list of areas, which correspond to the labels on a map. The RpgMap environment produces such a list, numbering and nesting the areas automatically.

```
\begin{ RpgMap } [<options>]
```

Begins a list of map areas, each of which is created with `\area`. Each area is given a heading, whose size depends on how deeply the map is nested: `\section*` at the top level, then `\subsection*`, and so on, down to `\subparagraph*`. These headings do not appear in the table of contents.

Areas are numbered according to their depth, cycling through numbers, capital letters and lower-case Roman numerals, so that the second area within the first area is 1B, and its third sub-area is 1B-iii. The following options are available:

Option	Effect	Default
<code>empty-prefix</code>	Text placed before the number of an unnamed area.	'Area '
<code>header-offset</code>	The number of levels by which to shrink the headings.	0
<code>prefix</code>	Text placed before the number of a named area.	'Area '
<code>title</code>	A title for the map, typeset as a heading one level larger than its areas. It is only used by the outermost map.	(empty)

```
\RpgMapRef { label }
```

`\RpgMapRef*` Typesets the name of the labelled area. The starred form typesets its number instead. This is a wrapper around `\ref` and `\nameref`.

```
\RpgMapRef{map:dais} is area
\RpgMapRef*{map:dais}, on
\RpgPage{map:dais}.
```

The Dais is area 1A, on page 24.

Inside a Map

Inside a map environment, the following commands and environments are valid.

`\area [<label>]{<name>}`

Adds an area to the current map, much as `\item` adds an item to a list. Its heading is ‘<prefix><number>: <name>’, or ‘<empty-prefix><number>’ if no `name` is given. Any text which follows forms the description of the area.

If a `label` is given, the area may be referred to with `\RpgMapRef` or `\RpgPage`.

`\begin{ RpgArea } [<label>]{<name>}`

Adds an area to the current map, and begins a nested map within it, so that the body of the environment may contain sub-areas. It is equivalent to an `\area` followed by an `RpgMap` environment, and raises an error if used outside of a map.

Nested Map Areas

```
\begin{RpgMap}[header-offset=2]
  \begin{RpgArea}[map:hall]{Great Hall}
    A long and draughty hall.

    \area[map:dais]{The Dais} A
      raised stone platform.

    \area An unremarkable alcove.
  \end{RpgArea}

  \area[map:cellar]{Cellar} Damp and
    dark.
\end{RpgMap}
```

Area 1: Great Hall

A long and draughty hall.

Area 1A: The Dais. A raised stone platform.

Area 1B. An unremarkable alcove.

Area 2: Cellar

Damp and dark.

RpgCards

Cards are small, playing-card sized boxes, useful for handouts such as items, spells or abilities. If the content of a card is too long to fit, it is automatically continued onto further cards.

Cards are rarely used directly: they are most often created by the rule environments (page 25), which can display the same content either as normal text or as a card.

RpgCard example

```
\begin{RpgCard}[vmargin=0.1cm, hmargin=0.1cm,
  height=5.2cm, width=3.6cm]
  \raggedright\footnotesize This is an
  example card with lots of
  text\footnote{And maybe a footnote too?}.
  \lipsum[2]
\end{RpgCard}
```

This is an example card with lots of text*. Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis.

Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

*And maybe a footnote too?

`\cardbreak {}`

Within an `RpgCard`, ends the current card and continues the text on the next, in the same way as `\pagebreak`. Outside of a card it has no effect, so it may safely be used in text which is only sometimes displayed as a card.

`\footnote { text }`

Within an `RpgCard`, `\footnote` is redefined so that its text appears at the end of the card’s contents, below a horizontal rule. **Only one footnote may appear in each card:** if more than one is used, only the last is shown. Card footnotes do not affect the document’s footnote numbering.

`\begin{ RpgCard } [ <options> ]`

Typesets its contents in one or more cards, placed side by side. The text is typeset in the `\RpgFontCardBody` (page 35) font element. The following options are available:

Option	Effect	Default
<code>cardsep</code>	The horizontal space between consecutive cards.	5pt
<code>color</code>	The colour of the card's background.	white
<code>footnote</code>	The symbol used to mark the card's footnote.	*
<code>height</code>	The outer height of each card.	9.7cm
<code>hmargin</code>	The horizontal space between the edge of the card and its text. It must be less than half of the <code>width</code> .	0.2cm
<code>opacity</code>	The opacity of the card's background.	1
<code>vcenter</code>	An alias of <code>vcentre</code> .	off
<code>vcentre</code>	If present, centres the text vertically within each card.	off
<code>vmargin</code>	The vertical space between the edge of the card and its text. It must be less than half of the <code>height</code> .	0.2cm
<code>width</code>	The outer width of each card.	6.75cm

Cards cannot be given a title, and cannot be nested within one another.

```
\begin{tikzpicture}%wrap in tikz so we
  can draw dimensions
\node[anchor=south west,inner
  sep=0pt,outer sep=0pt] at (0,0)
{
  \begin{RpgCard}[hmargin=0.5cm,
    vmargin=0.5cm, height=9.7cm,
    width=6.75cm]
    \subsubsection{Text goes here}
    It fits nicely into the card,
    wrapping over
    lines\footnote{We can also
      have a single footnote}.
  \end{RpgCard}\ignorespaces
};
%% Draw the dimensions on top; use macros
  to save space!
\heightLines{red}
\widthLines{ForestGreen}

\end{tikzpicture}
```

`\RpgCardReset {}`

Restores the default options of all subsequent cards to their original values.

`\RpgCardSet { options }`

Sets the default options of all subsequent cards. Options given to an individual `RpgCard` take priority over these defaults.

Rule Environments

The environments described so far are independent of any particular game system. Some elements, however, depend on the rules of the game being written for: a monster's statistics in one system look nothing like those in another. RPGTeX therefore provides four *rule environments*, whose properties and appearance are defined by the theme:

Environment	Purpose
RpgFeat	Feats, talents and other character features.
RpgItem	Items and equipment.
RpgSpell	Spells and similar abilities.
RpgStat	Statistics blocks for creatures and characters.

Each may be displayed either as normal text or as a card, controlled by a switch. Since the available properties depend on the theme, consult the documentation of the active theme for the details of each.

`\RpgGetKey` (*Variable*)

The value given to a property of the current rule environment, such as `\RpgGetKey{ rarity }`. It may only be used within a rule environment.

```
\begin{RpgItem}{<name>}[<key> = <value>]
  The value of the key is \RpgGetKey{<key>}.
\end{RpgItem}
```

```
\begin{ RpgIfEnv } { switch }[<options-on>]{ env-on }[<options-off>]{ env-off }
```

Begins the environment `env-on` (with `options-on`) if `switch` is on, and `env-off` (with `options-off`) otherwise. Any arguments which follow are passed to the chosen environment, so both environments must take the same arguments. This is the mechanism by which the rule environments switch between text and card modes, and it may be used to create similar switchable environments.

```
\begin{ Rpg[X] } [<card-options>]{ name }[<properties>]
```

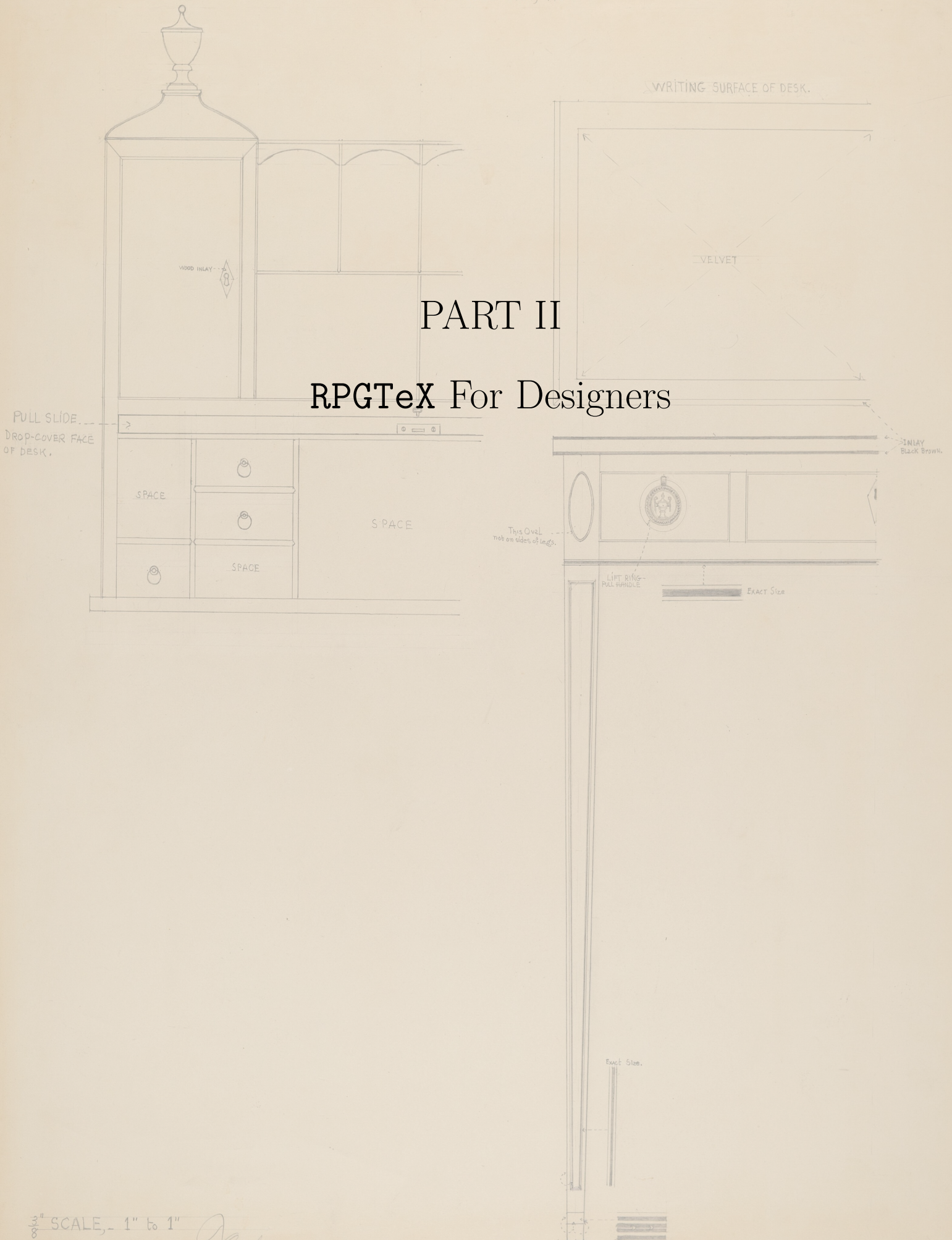
Typesets the rule element 'X' called **name**, with the given **properties** (a list of key-value pairs defined by the theme). Its display depends on the corresponding switch (page 17):

Environment (X)	Switch
RpgFeat	UseFeatCard
RpgItem	UseItemCard
RpgSpell	UseSpellCard
RpgStat	UseStatCard

If the switch is on, the element is displayed as an `RpgCard` (page 25), to which the `card-options` are passed. Otherwise (the default), it is displayed as normal text, and the `card-options` are ignored.

DESK.

METROPOLITAN MUSEUM OF ART, N.Y.C.



Chapter 6: Building a Theme

This part of the manual documents the commands used to build themes. They are intended for designers creating or modifying a theme, and for users who wish to adjust an existing theme in detail. By convention, all of these commands begin with `\RpgTheme....`

Authors of R_PG_TE_X documents probably do not need any of the commands in this part, and it can safely be ignored.

This chapter describes how a theme is organised. The chapters which follow describe how a theme defines its fonts (page 33), the design of its pages (page 36), the appearance of individual elements (page 38), and its rule environments (page 42).

Code Setters

Many of the commands in this part of the manual do not perform any action themselves. Instead, they *store* a piece of code, which R_PG_TE_X runs later, when the corresponding element is created. For example, `\RpgThemePartPage` stores the code used to draw a part page, which is run each time `\part` is called.

The stored code is often given arguments at the time it is run, such as the name of the part. These are accessed as `#1`, `#2` and so on within the stored code, even though the setter itself is called with only a single argument:

```
\RpgThemePartPage{The part is called #1}
```

The entry for each setter lists the arguments which are available.

Theme Files

When `\RpgSetTheme{ <name> }` is called, (page 12) R_PG_TE_X attempts to input a file of the form:

```
<theme path> / <name> / <name>.cfg
```

Where `<theme path>` is the value set by the theme path option (page 9) or by `\RpgSetThemePath` (page 12). The theme “`name`” is therefore defined by a directory, `<theme path>/name`, containing the file `name.cfg`. This file (the ‘theme configuration file’) may then execute any L^AT_EX code.

If the theme config file separates out the concerns across multiple files, it is important to `\input` them using the full file path to prevent name collisions. For example, the default theme’s configuration file begins:

```
\input{\RpgPackagePath/themes/default/colors.theme.tex}
\input{\RpgPackagePath/themes/default/utility.theme.tex}
...
```

Theme Code

Since themes are inserted entirely through `\input` commands, and may be introduced midway through a document, there are some restrictions and limitations on what should go inside a theme file or its subsidiary files:

1. **Packages.** Any package loading content (`\usepackage`, `\tcuselibrary`) should be wrapped in a `\RpgIfPreamble` (page 30) block, and appropriate guards put in place in case the packages cannot be loaded (because the theme was first activated in the document body).
2. **Other preamble-only commands.** Commands such as `\geometry` may also only be used in the preamble. Where a body equivalent exists, `\RpgIfPreambleTF` (page 29) can choose between the two (for example, `\geometry` in the preamble and `\newgeometry` in the body).
3. **Repeated loading.** A theme may be loaded more than once in the same document, either because the author switches back to it, or because another theme builds upon it: the `dnd` and `scifi` themes both begin by calling `\RpgSetTheme{default}`, and so the default theme is reloaded whenever either is activated. Every definition must therefore succeed when it is made a second time:

- Use `\DeclareDocumentCommand` (or `\cs_set:Npn`) rather than `\NewDocumentCommand` (or `\cs_new:Npn`), so that the definition is replaced rather than raising an error. This also allows a later theme to override a command defined by an earlier one.
 - Create variables only if they do not already exist (for example, with `\tl_if_exist:NF`), and use `\msg_set:nnn` rather than `\msg_new:nnn`.
4. **Category codes.** A theme file is read with the category codes in force where `\RpgSetTheme` is called. When it is called in the document body, L^AT_EX3 syntax is off and `@` is not a letter. Each file should therefore begin with `\ExplSyntaxOn` if it uses L^AT_EX3 syntax, and with `\makeatletter` if it uses internal commands such as `\@title`, rather than relying on the settings left by the file before it.
 5. **Timing.** A theme only affects material created after it is loaded. In particular, it cannot change the options passed to the underlying class (such as the font size or the number of columns), and it does not restyle a table of contents which has already been typeset.

Theme Resources

It is possible to ship non- $\text{\TeX}$ resources, such as fonts and images, with a theme. By convention, these are kept in subdirectories of the theme's directory, such as `fonts/` and `img/`.

$\text{\TeX}$, `graphicx` and `fontspec` search for these files relative to the document being compiled, not to the theme, and so each resource must be referred to by its full path. For example, the `dnd` theme sets its paper texture with:

```
\RpgThemePaperImage{\RpgPackagePath/themes/dnd/img/paper}
```

Note that this uses `\RpgPackagePath` because the theme is bundled with a package; for a general theme which is stored elsewhere, the designer must give the full path.

Fonts are loaded by giving `fontspec` the location of the font files. For example, the `scifi` theme loads its body font with:

```
\RpgIfFont{
  \providefontfamily{\jura}{Jura}[
    Path = \RpgPackagePath/themes/scifi/fonts/,
    UprightFont = *-Regular,
    ...
  ]
}
```

`\RpgThemeFont` (page 33) may be used unguarded, since it does not load any fonts itself, but the font declarations should be placed within `\RpgIfFont` (page 29), so that fonts are only loaded when the `font` option is active.

Conditional Code

Many parts of a theme are only meaningful when the corresponding package option is active, and some commands may only be used in the preamble, although a theme may be loaded in the document body. The following commands allow a theme to include code conditionally:

```
\RpgIfFontTF { true code }{ false code }
```

`\RpgIfFont` Runs `true code` if the `font` option is active (page 7), and `false code` otherwise. The alias `\RpgIfFont{ code }` takes a single argument, and is equivalent to `\RpgIfFontTF{ code }{}`.

```
\RpgIfPaperTF { true code }{ false code }
```

`\RpgIfPaper` Runs `true code` if the `background` option is active, and `false code` otherwise. The alias `\RpgIfPaper{ code }` takes a single argument, and is equivalent to `\RpgIfPaperTF{ code }{}`.

```
\RpgIfPreambleTF { true code }{ false code }
```


`\RpgIfPreamble` Runs `true` code if it is reached before `\begin{document}`, and `false` code otherwise. The alias `\RpgIfPreamble{ code }` takes a single argument, and is equivalent to `\RpgIfPreambleTF{ code }{ }`.
This allows a body alternative to be given for preamble-only commands, for example:

```
\RpgIfPreambleTF{\geometry{...}}{\newgeometry{...}}
```

The restrictions on what a theme may contain are described on page 28.

```
\RpgIfSectionsTF { true code }{ false code }
```

`\RpgIfSections` Runs `true` code if the `sections` option is active, and `false` code otherwise. The alias `\RpgIfSections{ code }` takes a single argument, and is equivalent to `\RpgIfSectionsTF{ code }{ }`.

```
\RpgIfTitleTF { true code }{ false code }
```

`\RpgIfTitle` Runs `true` code if the `title` option is active, and `false` code otherwise. The alias `\RpgIfTitle{ code }` takes a single argument, and is equivalent to `\RpgIfTitleTF{ code }{ }`.

Key-Value Options

Themes often need to give their own commands and environments key-value options. The `l3keys` library provides a convenient interface, but can be extremely verbose, and relies on excessive boilerplate to declare a new variable. `RPGTeX` provides commands which define such options in a single line each, creating the variable in which the value is stored if it does not already exist. Each key belongs to a **family**, such as `rpg/table`.

Defining Keys

Keys may be defined one at a time, giving the **family** each time (page 30), or several at once within a single **family** (page 31). The two sets of commands define exactly the same kinds of key.

Defining Individual Keys

Each of the following commands defines a single key in the given **family**:

```
\RpgThemeNewBoolKey (*){ family }{ key }{ variable }
```

Defines a flag, which takes no value. The unstarred form sets the boolean **variable** to false, and makes it true when the flag is present. The starred form does the opposite: the variable starts true, and the flag makes it false.

```
\RpgThemeNewCodeMapKey { family }{ key }{ choices }{ code }{ default }
```

Defines a key whose value must be one of the comma-separated **choices**. Rather than storing the value, it runs the corresponding entry of the comma-separated **code** list. Any entry of **code** which itself contains a comma must be enclosed in braces.

```
\RpgThemeNewTLChoiceKey { family }{ key }{ variable }{ choices }{ default }
```

Defines a key whose value must be one of the comma-separated **choices**, and stores it in **variable**.

```
\RpgThemeNewTLKey { family }{ key }{ variable }{ default }
```

Defines a key which takes any value, and stores it in **variable**. A value must be given whenever the key is used.

```
\RpgThemeNewTLMapKey { family }{ key }{ variable }{ choices }{ values }{ default }
```

Defines a key whose value must be one of the comma-separated **choices**, and stores the corresponding entry of the comma-separated **values** in **variable**. For example, the **columns** option maps the choices 1,2 to the values **onecolumn,twocolumn**.

Defining Groups of Keys

When several keys belong to the same **family**, they may be defined together, without repeating the family each time:

```
\RpgThemeKeys { family }{ definitions }
```

Defines several keys in the same family at once. The **definitions** are made with the commands below.

```
\RpgThemeKeys{rpg/my-env}
{
  \RpgThemeTLKey {title} {\l_myenv_title_tl} {Untitled}
  \RpgThemeBoolKey {hidden} {\l_myenv_hidden_bool}
  \RpgThemeTLChoiceKey {size} {\l_myenv_size_tl} {small,large} {small}
}

\keys_set:nn {rpg/my-env} {title=Hello, hidden, size=large}
```

The following commands may only be used within the definitions of `\RpgThemeKeys`, and define a key in its family:

```
\RpgThemeBoolKey (*){ key }{ variable }
```

Defines a flag, which takes no value. The unstarred form sets the boolean **variable** to false, and makes it true when the flag is present. The starred form does the opposite: the variable starts true, and the flag makes it false. The **family** is the one given to `\RpgThemeKeys`.

```
\RpgThemeCodeMapKey { key }{ choices }{ code }{ default }
```

Defines a key whose value must be one of the comma-separated **choices**. Rather than storing the value, it runs the corresponding entry of the comma-separated **code** list. Any entry of **code** which itself contains a comma must be enclosed in braces. The **family** is the one given to `\RpgThemeKeys`.

```
\RpgThemeTLChoiceKey { key }{ variable }{ choices }{ default }
```

Defines a key whose value must be one of the comma-separated **choices**, and stores it in **variable**. The **family** is the one given to `\RpgThemeKeys`.

```
\RpgThemeTLKey { key }{ variable }{ default }
```

Defines a key which takes any value, and stores it in **variable**. A value must be given whenever the key is used. The **family** is the one given to `\RpgThemeKeys`.

```
\RpgThemeTLMaKey { key }{ variable }{ choices }{ values }{ default }
```

Defines a key whose value must be one of the comma-separated **choices**, and stores the corresponding entry of the comma-separated **values** in **variable**. For example, the **columns** option maps the choices 1,2 to the values **onecolumn,twocolumn**. The **family** is the one given to `\RpgThemeKeys`.

Setting Keys

Once a key has been defined, its value is set using either the `l3keys` function `\keys_set:nn{ family }{ key-value-list }` (which requires `\ExplSyntaxOn`), or the `LATEX` kernel command `\SetKeys[<family>]{ key-value-list }`, which may

be used anywhere. The **family** is optional for `\SetKeys`, but should always be given: if it is omitted, the family of the package currently being loaded is used instead.

The new values are local to the group in which they are set (except where the code run by a `\RpgThemeNewCodeMapKey` key makes a global change), so they revert at the end of the group. Every environment is a group, and a group may also be created with braces, `\begin group` and `\end group`, or `\group_begin:` and `\group_end:`.

Chapter 7: Fonts

RPGTeX allows a high degree of customisation of the fonts and typefaces used throughout a document. Rather than setting fonts directly, each part of the document (headings, table text, footers and so on) draws its font from a named *font element*, such as `\RpgFontSection` (page 34). The theme defines the appearance of each element using `\RpgThemeFont`, and assigns each element to the relevant part of the document. Changing the appearance of an element therefore changes it consistently wherever that element is used.

Why didn't my font change?

`\RpgThemeFont` does not change any fonts directly: it updates the stored definition of a *font element*, which a theme designer must then assign to the relevant part of the document. For example, the `\RpgFontSection` (page 34) element does not ‘hook in’ to the `\section` command by itself; it changes the section font only because most themes call `\titleformat{\section}{\RpgFontSection}...`. If the `\titleformat` were later changed (for whatever reason), the section font would no longer update.

If an element does not change font after its definition is updated, first check that the font element is actually being invoked – and if not, invoke it manually. Once the command is in place, the font will change as expected.

Note also that `\RpgThemeFont` has no effect unless the `font` option is active (page 7). This is the default when using one of the RPGTeX classes.

Font Commands

`\RpgThemeFont { key-value-list }`

Updates the font elements specified by a comma-separated list of key-value pairs. Each element has two keys, `<name>-family` and `<name>-style`, where `<name>` is given in the ‘Components’ column of the table below (or was assigned by `\RpgThemeCreateFont`).

The values may be any L^AT_EX font or formatting commands (such as `\slshape\scriptsize\bfseries`), which are applied whenever the element is invoked. To change the typeface itself, a font family must first be defined, for example using `\newfontfamily` from the `fontspec` package.

Font Interface

```
\subsection{The Original Font}
Here is some text

\newfontfamily{\courier}{Courier}
\RpgThemeFont{main-body-family=\courier,
  subsection-style=\slshape\Huge}

\subsection{The New Font}
And after the change is introduced
```

The Original Font

Here is some text

The New Font

And after the change is introduced

`\RpgThemeCreateFont { \ElementName }{ key-base }`

Creates a new font element, `\ElementName`, along with the keys `<key-base>-family` and `<key-base>-style`. A new element is initially blank (it applies no formatting) until it is assigned a value using `\RpgThemeFont`. Thereafter, it may be used and modified in exactly the same way as the built-in font elements.

Create Font

```

\RpGThemeCreateFont{\MyFont}{myfont}
\newfontfamily{\courier}{Courier}
\RpGThemeFont{
  myfont-family = \courier,
  myfont-style = \slshape
}

This is text

{\MyFont This is text in my new font}

```

This is text

This is text in my new font

Font Elements

RPgTeX provides 35 font elements by default (themes may define more). Since each element is assigned to part of the document by the theme designer, an element intended for one purpose (such as section headings) may be used for a different purpose within another theme. The descriptions below therefore reflect how the elements are used by RPgTeX itself and by the provided themes.

Family vs Style

Each font element is defined by two components: a **family** and a **style**. Formally, the **family** defines the **typeface** used by the element, whilst the **style** determines the options applied to that typeface (weight, shape, size, colour and so on).

In practice the distinction is largely one of clarity, since the font element simply applies the two in sequence:

```
\RpGFontX = <x-family> <x-style>
```

It is therefore generally safe to place commands which belong in the family within the style key, provided that they do not conflict with other styling.

Font vs Implementation

We encourage designers to place all text styling within the relevant font element, rather than elsewhere. If all subsections are to be red, then set `subsection-style=\color{ red }`, rather than setting the colour within the `titlesec` specification.

Font Element	Components	Usage
\RpGFontBody	main-body-family main-body-style	The main body text of the document. \RpGThemeFont sets \normalfont equal to this element. Changing the font size here (e.g. with \large) can give counterintuitive results, since it affects <i>only</i> the body text and does not scale the other elements accordingly. The font size of the whole document should instead be set using the <code>size</code> option (page 7).
\RpGFontEmphasis	emph-family emph-style	The font used by \RpGKey, and therefore by \emph unless the <code>noemph</code> option is passed.
\RpGFontTitle	title-family title-style	The font used for the document title, as returned by \RpGGetTitle and displayed by \maketitle.
\RpGFontSubTitle	subtitle-family subtitle-style	The font used for the subtitle and author, as returned by \RpGGetSubtitle and \RpGGetAuthor and displayed by \maketitle.
\RpGFontPart	part-family part-style	The font used for the part name on the part page created by \part.
\RpGFontChapter	chapter-family chapter-style	The font used for \chapter headings.
\RpGFontSection	section-family section-style	The font used for \section headings.
\RpGFontSubsection	subsection-family subsection-style	The font used for \subsection headings.
\RpGFontSubsubsection	subsubsection-family subsubsection-style	The font used for \subsubsection headings.

\RpgFontParagraph	paragraph-family paragraph-style	The font used for \paragraph headings.
\RpgFontSubparagraph	subparagraph-family subparagraph-style	The font used for \subparagraph headings.
\RpgFontTableTitle	table-title-family table-title-style	The title of an RpgTable (page 20), if one is set using the title option.
\RpgFontTableHeader	table-header-family table-header-style	The first (header) row of an RpgTable (page 20).
\RpgFontTableBody	table-body-family table-body-style	The rows of an RpgTable (page 20) after the header row.
\RpgFontTipTitle	tip-title-family tip-title-style	The title of an RpgTip (page 19) environment.
\RpgFontTipBody	tip-body-family tip-body-style	The body of an RpgTip (page 19) environment.
\RpgFontSidebarTitle	sidebar-title-family sidebar-title-style	The title of an RpgSidebar (page 19) environment.
\RpgFontSidebarBody	sidebar-body-family sidebar-body-style	The body of an RpgSidebar (page 19) environment.
\RpgFontNarration	narration-family narration-style	All text within an RpgNarration (page 19) environment (which has no title).
\RpgFontAbstractTitle	abstract-title-family abstract-title-style	The title of an RpgAbstract (page 21) environment.
\RpgFontAbstractBody	abstract-body-family abstract-body-style	The body of an RpgAbstract (page 21) environment.
\RpgFontSecretTitle	secret-title-family secret-title-style	The labels which distinguish the hidden and public text of an RpgSecret (page 22) environment when secrets are shown.
\RpgFontCardTitle	card-title-family card-title-style	The title of a card, as used by the card-switching environments of the provided themes (such as RpgItem).
\RpgFontCardHeader	card-header-family card-header-style	Subheadings and descriptive headers within a card (such as the type and rarity of an RpgItem).
\RpgFontCardBody	card-body-family card-body-style	The body text of an RpgCard (page 25).
\RpgFontTocPart	toc-part-family toc-part-style	Part entries in the table of contents.
\RpgFontTocChapter	toc-chapter-family toc-chapter-style	Chapter entries in the table of contents.
\RpgFontTocSection	toc-section-family toc-section-style	Section entries in the table of contents.
\RpgFontStatBlockTitle	stat-block-title-family stat-block-title-style	The title of a statblock environment (in the dnd theme, RpgStat).
\RpgFontStatBlockBody	stat-block-body-family stat-block-body-style	The body text of a statblock environment.
\RpgFontStatBlockSection	stat-block-section-family stat-block-section-style	Section headings within a statblock environment.
\RpgFontFooter	footer-family footer-style	The footer text.
\RpgFontPageNumber	page-number-family page-number-style	The page number within the footer.
\RpgFontDropCap	drop-cap-family drop-cap-style	The large initial letter created by \RpgDropCap.
\RpgFontDropCapAfter	drop-cap-after-family drop-cap-after-style	The text which follows the initial letter of an \RpgDropCap, on the first line.

Chapter 8: Page Design

This chapter describes the commands which define the overall design of a theme's pages: their backgrounds and footers, and the title and part pages.

Backgrounds & Footers

These commands control the page background and footer. Their effects depend on the **bg** option (page 7): the paper colour and image are only shown when it is **full**, and the footer decorations unless it is **none**. The paper and footer commands have no effect unless the **background** option is active.

The footer commands draw on the page using the **eso-pic** package. Positions are measured from the bottom-left corner of the page, and further content may be placed with **\RpgPut** (page 17).

```
\RpgThemeFooterDecoration (*) { height } { path/to/image } { overlay }
```

Draws an image across the full width of the bottom of the current page, with the given **height**, and then draws the **overlay** (such as **\RpgPut** or **\RpgThemePageNumberAt** commands) on top of it.

The decoration is only drawn on the page on which the command is called, and so it is normally placed within the definition of a page footer. The starred form mirrors the image horizontally, and the unstarred form draws it as it is, allowing a single image to be used on both left- and right-hand pages:

```
\fancyfoot[LE]{%
  \RpgThemeFooterDecoration{1.5cm}{path/to/scroll}{
    \RpgPut{0.43in}{0.15in}{\RpgFontPageNumber \thepage}
  }
}
\fancyfoot[RO]{%
  \RpgThemeFooterDecoration*{1.5cm}{path/to/scroll}{
    \RpgPut{\paperwidth-0.43in}{0.15in}{\RpgFontPageNumber \thepage}
  }
}
```

```
\RpgThemePageNumberAt { x } { y }
```

Typesets the page number, in the **\RpgFontPageNumber** (page 35) font element, centred at the position (x, y). It may only be used within a page overlay, such as the final argument of **\RpgThemeFooterDecoration**.

```
\RpgThemePaperColor { colour }
```

Sets the background colour of every page, and updates **PaperColor** to match (page 9).

```
\RpgThemePaperImage { path/to/image }
```

Sets an image, stretched to fill the page, as the background of every subsequent page. This is typically a paper texture. An empty path removes any existing background image.

Title & Part Pages

These commands define the appearance of the pages created by **\maketitle** (page 13) and **\part** (page 13). The cover and part page designs are drawn on the page with **eso-pic**, usually with **\RpgPut** (page 17), while the

header title is typeset as ordinary text. Within them, the title information is available through `\RpgGetTitle`, `\RpgGetSubtitle` and `\RpgGetAuthor` (page 12).

`\RpgThemeOnPartChange { code }`

Stores `code` to be run after every `\part` (in either form). For example, the `dnd` theme uses this to cycle its colour scheme with each new part.

`\RpgThemePartPage { code }`

Stores the design of the part page created by `\part`, which is drawn on top of the part's background image (if any). The available arguments are:

Argument	Value
----------	-------

#1	The name of the part.
----	-----------------------

The part number is available through `\thepart`. By default, the part number and name are centred on the page in the `\RpgFontPart` (page 34) font element.

`\RpgThemeTitleCover { code }`

Stores the design of the cover page created by `\maketitle` in `cover` mode. It is drawn on top of the `\cover` image, if one has been set, or on an otherwise blank page.

`\RpgThemeTitleHeader { code }`

Stores the code which typesets the title when `\maketitle` is called in `header` mode. Unlike the cover, this is ordinary text, placed at the point where `\maketitle` is called.

`\RpgThemeTitleMode { cover / header }`

Selects which of the two designs `\maketitle` produces. Any other value raises an error. The `rpgbook` class selects `cover`; otherwise, the default is `header`.

Chapter 9: Styling Elements

Box Formatters

The appearance of most environments is controlled through `tcolorbox` styles, which a theme may redefine with `\tcbset`:

```
\tcbset{rpgsidebar/.style={colframe=black, ...}}
```

The available styles are:

Style	Used by
<code>rpgcard</code>	The <code>RpgCard</code> (page 25) environment.
<code>rpgnarration</code>	The <code>RpgNarration</code> (page 19) callout.
<code>rpgsecret</code>	The box used by <code>RpgSecret</code> (page 22).
<code>rpgsidebar</code>	The <code>RpgSidebar</code> (page 19) callout.
<code>rpgtip</code>	The <code>RpgTip</code> (page 19) callout.

Redefining a style with `/.style` replaces it entirely, including the theme's settings. To adjust a style instead, add to it with `/.append style`:

Restyling a Callout

```
\tcbset{
  rpgtip/.append style={
    sharp corners,
    borderline west={3pt}{0pt}{black},
  }
}

\begin{RpgTip}{A Restyled Tip}
  This tip has square corners, and a rule
  down its left-hand side.
\end{RpgTip}
```

A Restyled Tip

This tip has square corners, and a rule down its left-hand side.

Other Formatters

The following commands control the elements which are not built from a `tcolorbox` style.

```
\RpgThemeDiceFormat { code }
```

Stores the code which typesets a dice expression, after it has been parsed by `\RpgDice` (page 15). The available arguments are:

Argument	Value
<code>#1</code>	The number of dice (empty if none was given).
<code>#2</code>	The number of sides.
<code>#3</code>	The evaluated modifier, including its sign (for example, <code>+6</code> or <code>-2</code>).

By default, the expression is typeset as `#1d#2#3`.

The arguments are passed as variables which contain these values, rather than as the values themselves. They typeset correctly as they are, but must be expanded before being tested or compared: for example, with `\tl_if_blank:eTF` rather than `\tl_if_blank:nTF`.

RpgDice Formatting: The D&D Format

Before: `\RpgDice{2d8 + 3}, \RpgDice{d8}`

```
\ExplSyntaxOn
\RpgThemeDiceFormat{ % #1: Dice Number,
  #2: Dice Sides, #3: Modifier
  %% d8 -> 1d8
  %% the arguments are variables, so
  %% must be expanded before testing
  \tl_set:Nx \l_tmpa_tl {
    \tl_if_blank:ETF {#1} {1} {#1} }
  %% compute the average roll, and
  %% print it first
  \fp_eval:n { floor ( \l_tmpa_tl * (
    #2 + 1 ) / 2 ) + ( #3 ) }
  ~( \l_tmpa_tl d#2 \fp_compare:nNnF {
    #3 } = { 0 } { #3 } )
}
\ExplSyntaxOff
```

After: `\RpgDice{2d8 + 3}, \RpgDice{d8}`

Before: 2d8+3, d8

After: 12(2d8+3), 4(1d8)

\RpgThemeDropCapHeight { lines }

Sets the number of lines spanned by the initial letter of `\RpgDropCap` (page 16). The default is 4.

Changing the Drop Cap Height

`\RpgThemeDropCapHeight{2}`

`\RpgDropCap{T}{he road}` winds down from the pass into a valley of grey stone, where the river has long since run dry and the old watchtowers stand empty.

The road winds down from the pass into a valley of grey stone, where the river has long since run dry and the old watchtowers stand empty.

\RpgThemeMapFormat { code }

Stores the code which formats one part of the number of an area within an `RpgMap` (page 23). The number of an area is built by running this code once for each level of nesting, from the outermost map inwards, and joining the results. The available arguments are:

Argument	Value
#1	The depth being formatted (1 for the outermost map).
#2	The position of the area at that depth (1 for the first area).

For example, the third area inside the second area of a map is numbered by running the code with `{1}{2}`, and then with `{2}{3}`. By default, the depths cycle through numbers, capital letters, and lower-case Roman numerals preceded by a hyphen, giving numbers such as 1B-iii.

The code is run during full expansion, and so must be expandable: functions such as `\int_to_Roman:n` may be used, but assignments may not.

RpgThemeMapFormat: Roman Numerals

```
\ExplSyntaxOn
\RpgThemeMapFormat{ % #1: depth, #2:
  position at that depth
  \int_compare:nNnTF {#1} = {1}
  { \int_to_Roman:n {#2} } %%
    outermost: I, II, III, ...
  { . \int_to_arabic:n {#2} } %%
    deeper: .1, .2, .3, ...
}
\ExplSyntaxOff

\begin{RpgMap}[header-offset=2]
  \area{Gatehouse} The only way in.

  \begin{RpgArea}{Keep}
    \area{Armoury} Racks of old
    spears.

    \begin{RpgArea}{Cellars}
      \area{Wine Store} Mostly
      empty.
    \end{RpgArea}
  \end{RpgArea}
\end{RpgMap}
```

Area I: Gatehouse

The only way in.

Area II: Keep

Area II.1: Armoury. Racks of old spears.

Area II.2: Cellars.

Area II.2.1: Wine Store. Mostly empty.

\RpgThemeSetFiligree { code }

Stores the tikz code which draws the filigree frame (page 21). It is drawn over the box, with `FiligreeColor` as the line colour, and may refer to the box's corners through the `frame` node (such as `frame.north west`). The size of the design should be scaled by `\RpgFiligreeSize` (page 10).

A Custom Filigree

```
\RpgThemeSetFiligree{
  %% a bracket at each corner, scaled
  by \RpgFiligreeSize
  \foreach \corner/\dx/\dy in
    {north west/1/-1, north
     east/-1/-1, south west/1/1,
     south east/-1/1}
  {
    \draw[thick]
      ($ (frame.\corner)+(0,{2*\dy*\RpgFiligreeSize})
      -- (frame.\corner)
      --
      ($ (frame.\corner)+({2*\dx*\RpgFiligreeSize},0)$);
  }
}

\begin{RpgFiligree}
  The frame is drawn over the box, so
  it can extend beyond its edges.
\end{RpgFiligree}
```

The frame is drawn over the box, so it can extend beyond its edges.

\RpgThemeSetSecretHidden { code }

Stores the code which typesets an `RpgSecret` while secrets are hidden. The available arguments are:

Argument	Value
#1	The public text (possibly empty).
#2	The secret text.
#3	Any options given to the environment which are not recognised by it, intended to be passed to a <code>tcolorbox</code> .

By default, only the public text (#1) is typeset.

Stores the code which typesets an `RpgSecret` while secrets are shown. The available arguments are the same as for `\RpgThemeSetSecretHidden`. By default, the secret text is typeset in a `tcolorbox` with the `rpgsecret` style, with the public text (if any) in its lower part.

The following example sets both commands, so that secrets appear inline, and are revealed in brackets after the public text:

Inline Secrets

```
\RpgThemeSetSecretHidden{#1}  
\RpgThemeSetSecretVisible{#1  
  \textcolor{SecretColor!60!black}{[#2]}}  
  
\RpgSwitch{ShowSecrets}{false}  
Players: the door \begin{RpgSecret}{is  
  locked}is trapped\end{RpgSecret}.  
  
\RpgSwitch{ShowSecrets}{true}  
GM: the door \begin{RpgSecret}{is  
  locked}is trapped\end{RpgSecret}.
```

Players: the door is locked.
GM: the door is locked [is trapped].

Chapter 10: Rule Environments

Rule environments (page 25) are the elements of a document which depend on the rules of a particular game, such as items, spells and monsters. `RPGTeX` itself only creates them: the theme decides which properties each one accepts, and how it is displayed. `RPGTeX` creates four rule environments, `RpgItem`, `RpgSpell`, `RpgFeat` and `RpgStat`, each of which may be displayed either as text or as a card. Designers may also create their own.

Defining a rule environment has three steps:

1. **Create** the environment, with `\RpgThemeNewRuleEnv` or `\RpgThemeNewCardableRuleEnv`. This is only necessary for new environments, since the four standard ones already exist.
2. **Add properties** to it, with the `\Rpg[X]AddProperty` and `\Rpg[X]AddBoolean` commands generated for each environment.
3. **Set its format**, with the `\RpgTheme[X]Format` (and, for a card, `\RpgTheme[X]CardFormat`) command generated for each environment.

Throughout this chapter, `[X]` stands for the name of the environment, so that the commands for `RpgItem` are `\RpgItemAddProperty`, `\RpgThemeItemFormat` and so on.

Creating a Rule Environment

```
\RpgThemeNewCardableRuleEnv{Potion}{UsePotionCard}
\RpgPotionAddProperty{colour}{\PotionColour}{clear}

\RpgThemePotionFormat{
  \paragraph{#1} (\PotionColour) #2
}
\RpgThemePotionCardFormat{
  {\RpgFontCardTitle #1}\par
  \textit{A \PotionColour{} liquid}\par
  #2
}

\begin{RpgPotion}{Healing Draught}[colour=red]
  Restores \RpgDice{2d4+2} hit points.
\end{RpgPotion}

\RpgSwitch{UsePotionCard}{on}
\begin{RpgPotion}[width=4.5cm,
  height=3cm]{Healing Draught}[colour=red]
  Restores \RpgDice{2d4+2} hit points.
\end{RpgPotion}
\RpgSwitch{UsePotionCard}{off}
```

Healing Draught. (red) Restores 2d4+2 hit points.

Healing Draught

A red liquid
Restores 2d4+2 hit points.

Creating Rule Environments

`\RpgThemeNewCardableRuleEnv` [`<prefix>`]{ `name` }{ `switch` }[`<card-options>`][`<properties>`]

Creates the environment `<prefix><name>` (the prefix is `Rpg` by default), which may be displayed either as text or as a card, together with a new switch (page 17) which chooses between them. The switch is off (text mode) by default. The environment is used as:

```
\begin{<prefix><name>}[<card-options>]{<title>}[<properties>]
  <body>
\end{<prefix><name>}
```

In card mode, the `card-options` are passed to `RpgCard` (page 25); in text mode they are ignored. The optional `card-options` and `properties` given to `\RpgThemeNewCardableRuleEnv` are the defaults used when the corresponding argument is omitted. They are replaced, rather than combined, when the argument is given.

In addition to the commands generated by `\RpgThemeNewRuleEnv`, this creates `\RpgTheme[X]CardFormat`.

Creating an environment which already exists redefines it, resetting its formats to their defaults; an existing switch keeps its current value.

```
\RpgThemeNewRuleEnv [<prefix>]{ name }
```

Creates the environment `<prefix><name>` (the prefix is `Rpg` by default), which is always displayed as text. The environment is used as:

```
\begin{<prefix><name>}{<title>}[<properties>]
  <body>
\end{<prefix><name>}
```

It also creates the commands `\Rpg[X]AddProperty`, `\Rpg[X]AddBoolean` and `\RpgTheme[X]Format`, using the same prefix.

Creating an environment which already exists redefines it, resetting its format to the default.

Properties

Each property is a key which may be given in the `properties` argument of the environment. Its value is stored in a variable, which the format can then use. Properties are set separately for each use of the environment, so a property which is not given takes its default value.

Properties belong to the environment, not to the theme which added them. Adding a property with the same name again replaces it, but a property added by one theme remains available after another theme is loaded.

```
\Rpg[X]AddBoolean (*){ key }[<aliases>]{ variable }
```

Adds a flag property, which takes no value. The unstarred form sets the boolean `variable` to false, and makes it true when the flag is given; the starred form does the opposite. The optional `aliases` are a comma-separated list of alternative names for the same flag.

```
\Rpg[X]AddProperty { key }[<aliases>]{ variable }{ default }
```

Adds a property which takes a value, stored in `variable`, with the given `default`. The optional `aliases` are a comma-separated list of alternative names for the same property.

Within either format, the value of a property may also be retrieved by its name with `\RpgGetKey` (page 26). This does not work for flag properties, whose variables should be tested directly (for example, with `\bool_if:NTF`).

Formats

The format of a rule environment is a code setter (see the sidebar (page 28)). Both formats have the same arguments:

Argument	Value
#1	The title of the element.
#2	The body of the environment.

The values of the properties are available through their variables (or `\RpgGetKey`). The following examples restyle the `RpgPotion` environment created at the start of this chapter; a format set within an example applies only to that example.

Stores the code which typesets the element in card mode. It is typeset inside an `RpgCard` (page 25), and so continues onto further cards if it is too long for one. By default, the title is typeset as a `\subsection`, followed by the body.

A Custom Card Format

```
\RpgPotionAddProperty{colour}{\PotionColour}{gray}
\RpgPotionAddProperty{rarity}{\PotionRarity}{common}

\RpgThemePotionCardFormat{
  \begin{center}
    {\RpgFontCardTitle #1}\par
    \textcolor{\PotionColour!70!black}{\rule{0.5\linewidth}{1pt}}\par\medskip
    \vfil

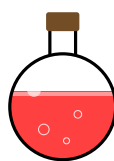
    \begin{tikzpicture}
      %% the liquid, filling the lower
      %% part of the flask
      \begin{scope}
        \clip (0,0) circle (0.7);
        \fill[\PotionColour!75]
          (-0.7,-0.7) rectangle
          (0.7,0.2);
        \fill[\PotionColour!40]
          (-0.7,0.15) rectangle
          (0.7,0.2);
        \foreach \x/\y/\r in
          {-0.25/-0.3/0.07,
           0.2/-0.1/0.05, 0.05/-0.45/0.04}
          { \draw[white, opacity=0.8]
            (\x,\y) circle (\r); }
      \end{scope}
      %% the flask, its neck and the cork
      \draw[very thick] (-0.17,0.68) --
        (-0.17,1.05) (0.17,0.68) --
        (0.17,1.05);
      \draw[very thick] (0,0) circle (0.7);
      \fill[brown!60!black, rounded
        corners=1pt] (-0.22,1.0) rectangle
        (0.22,1.25);
      \fill[white, opacity=0.6]
        (-0.38,0.22) circle (0.09);
    \end{tikzpicture}\par\medskip
    \textsc{\PotionRarity}\par\medskip
    \vfil

    \small\itshape #2

    \vfil
  \end{center}
}

\RpgSwitch{UsePotionCard}{on}
\begin{RpgPotion}[width=5cm, height=6cm]{Potion
of Healing}[colour=red, rarity=uncommon]
  Restores \RpgDice{2d4+2} hit points when
  drunk.
\end{RpgPotion}
\begin{RpgPotion}{Flask of Oil}[colour=orange]
  When thrown or broken, covers all creatures
  and objects within 5ft in oil. The next
  time a creature covered in oil takes fire
  damage, they take \RpgDice{3d8}
  additional fire damage.
\end{RpgPotion}
\RpgSwitch{UsePotionCard}{off}
```

Potion of Healing



UNCOMMON

Restores 2d4+2 hit points when drunk.

Flask of Oil



COMMON

When thrown or broken, covers all creatures and objects within 5ft in oil. The next time a creature covered in oil takes fire damage, they take 3d8 additional fire damage.

Stores the code which typesets the element in text mode. By default, the title is typeset as a `\subsection`, followed by the body.

A Custom Text Format

```
\RpgPotionAddProperty{colour}{\PotionColour}{gray}
\RpgPotionAddProperty{rarity}{\PotionRarity}{common}
\RpgPotionAddBoolean{magic}{\PotionMagic}

\ExplSyntaxOn
\RpgThemePotionFormat{ % #1: title, #2: body
  \begin{tcolorbox}[
    enhanced, sharp~corners,
    boxrule=0pt, leftrule=4pt,
    colframe=\PotionColour!70!black,
    colback=\PotionColour!7,
    colbacktitle=\PotionColour!70!black,
    coltitle=white,
    title={
      \textbf{#1}
      \bool_if:NT \PotionMagic
      {
        \hspace{0.4em}
        \tikz[baseline=-0.6ex]
        \fill[yellow!80!orange]
          (0,4pt) -- (1pt,1pt) --
          (4pt,0) -- (1pt,-1pt)
          -- (0,-4pt) --
          (-1pt,-1pt) -- (-4pt,0)
          -- (-1pt,1pt) -- cycle;
      }
      \hfill \textit{\PotionRarity}
    },
  ],
  #2
  \end{tcolorbox}
}
\ExplSyntaxOff

\begin{RpgPotion}{Potion of
  Healing}[colour=red, rarity=uncommon, magic]
  Restores \RpgDice{2d4+2} hit points when
  drunk.
\end{RpgPotion}
\begin{RpgPotion}{Flask of Oil}[colour=orange]
  When thrown or broken, covers all creatures
  and objects within 5ft in oil. The next
  time a creature covered in oil takes fire
  damage, they take \RpgDice{3d8}
  additional fire damage.
\end{RpgPotion}
```

Potion of Healing ✨ *uncommon*

Restores 2d4+2 hit points when drunk.

Flask of Oil *common*

When thrown or broken, covers all creatures and objects within 5ft in oil. The next time a creature covered in oil takes fire damage, they take 3d8 additional fire damage.

A close-up, low-angle photograph of a vintage typewriter keyboard. The keys are made of brass and are arranged in a standard QWERTY layout. The keys in the foreground are in sharp focus, while the keys in the background are blurred. The lighting is warm and directional, coming from the upper right, which creates highlights on the top surfaces of the keys and casts soft shadows. The overall tone is sepia or aged, giving it a historical feel. The text "PART III" and "Appendices" is centered over the middle of the keyboard.

PART III

Appendices

Appendix A: The rpglatex Compiler

`rpglatex` is a Python 3 script (`scripts/rpglatex`) which wraps `xelatex` or `lualatex`, adding several conveniences for compiling R_PG_TE_X documents. To use it from any directory, place it on your `PATH`, for example with a symbolic link:

bash

```
ln -s ~/path/to/rpgtex/scripts/rpglatex ~/.local/bin/rpglatex
```

`rpglatex` [options] <file>

Compiles <file>, performing the following steps automatically:

- Defines `\RpgPackagePath`, so that R_PG_TE_X does not need to locate itself.
- Runs the compiler twice, so that references and `tikz remember picture` commands resolve correctly (three times with `-full`).
- Runs `bibtex` between the passes, if requested with `-ref`, and `makeindex` whenever the document produces an index.
- Keeps the auxiliary files (`.aux`, `.log` and so on) in a separate build directory, and moves only the finished PDF to the output location.
- Hides the compiler's lengthy output, printing a short summary instead. If compilation fails, the compiler's output is printed so that the error can be found.

The following options are available:

Option	Effect	Default
<code>-b, -build <dir></code>	The directory in which auxiliary files are stored.	<code>.build</code>
<code>-f, -full</code>	Runs the compiler three times, rather than twice, for very complex or reference-heavy documents.	off
<code>-l, -lualatex</code>	Compiles with <code>lualatex</code> .	off
<code>-o, -output <name></code>	The name of the finished PDF.	<code><file>.pdf</code>
<code>-p, -print</code>	Compiles in print mode, by setting <code>bg=print</code> through the command line interface (page 10).	off
<code>-q, -quiet</code>	Suppresses the script's own progress messages.	off
<code>-r, -ref</code>	Runs <code>bibtex</code> after the first pass.	off
<code>-show <0/1></code>	Whether to open the finished PDF (or switch to it, if already open) in a viewer.	1
<code>-v, -verbose</code>	Shows the compiler's full output.	off
<code>-x, -xelatex</code>	Compiles with <code>xelatex</code> .	on

For example, the following compiles a print version of `adventure.tex` with `lualatex`, without opening it:

```
rpglatex adventure.tex -l -p -show 0
```

Appendix B: Dependencies

RPGTeX loads the packages listed below. All of them are included in a full TeX Live installation, and MiKTeX installs missing packages automatically the first time they are needed. On a minimal installation, missing packages may be added with `tlmgr`:

```
bash
```

```
tlmgr install <package>
```

Some files are distributed as part of a larger package, whose name must be given to `tlmgr` instead; this name is shown in brackets. Packages marked with an asterisk are only loaded in some circumstances, which are described beneath the list.

<code>adjustbox</code>	<code>graphicx</code> (graphics)	<code>stfloats</code> (sttools)
<code>array</code> (tools)	<code>hang</code>	<code>supertabular</code>
<code>contour</code>	<code>l3keys2e</code> (l3packages)	<code>tabularx</code> (tools)
<code>cuted</code> (sttools)	<code>lettrine</code>	<code>tcolorbox</code>
<code>enumitem</code>	<code>luacolor</code> *	<code>tikz</code> (pgf)
<code>eso-pic</code>	<code>magaz</code>	<code>titlesec</code>
<code>extarticle</code> * (extsizes)	<code>microtype</code>	<code>titletoc</code> (titlesec)
<code>extbook</code> * (extsizes)	<code>multitoc</code> * (ms)	<code>tocloft</code>
<code>fancyhdr</code>	<code>nameref</code> (hyperref)	<code>transparent</code>
<code>float</code>	<code>ragged2e</code>	<code>xcolor</code>
<code>fontenc</code> * (latex)	<code>relsize</code>	<code>xltabular</code>
<code>fontspec</code> *	<code>Royal</code> (cfr-initials)	<code>xparse</code> (l3packages)
<code>geometry</code> *	<code>standalone</code> *	

The packages marked with an asterisk are loaded as follows:

- `extarticle` and `extbook`: as the base classes of `rpghandout` and `rpgbook` respectively.
- `fontenc` and `fontspec`: when the `font` option is active (page 7).
- `geometry`: by the RPGTeX classes.
- `luacolor`: when compiling with `lualatex`.
- `multitoc`: by the RPGTeX classes, unless the `nomultitoc` option is used.
- `standalone`: by the `rpgcard` and `rpgdeck` classes.

The `rpgcard` and `rpgdeck` classes are built on the standard `article` class, which is part of every L^AT_EX installation, as is `expl3`.

Fonts

The `default` and `dnd` themes use fonts from the T_EX distribution, while the fonts of the `scifi` theme are distributed with RPGTeX itself, in `themes/scifi/fonts`. Fonts are only loaded when the `font` option is active. The `default` theme uses Latin Modern, which is part of every T_EX distribution; the `dnd` theme uses:

- `texgyrebonum-*.otf` (`tex-gyre`): TeX Gyre Bonum, the main body font.
- `EBGaramond-Regular.otf` (`ebgaramond`): EB Garamond, used for small capitals.
- `KpSans-*.otf` and `KpRoman-*.otf` (`kpfonts-otf`): Kp-Fonts, used for titles and table headings.
- `GilliusADFN02-*.otf` (`gillius`): Gillius ADF No2, used for subtitles and table text.

Appendix C: Image Credits

The following images are used in this document. We are grateful to their creators, and to the organisations which make them freely available.

Typewriter Used on the cover and on the title page of the Appendices.

Keyboard, Typewriter, Antiquity by [bjoern-hiekel](#), from [Pixabay](#). Used under the Pixabay Content License.

The Writing Desk Used on the title page of Part I.

Childe Hassam, *The Writing Desk* (1915), etching in black on wove paper. National Gallery of Art, Washington, Corcoran Collection (Bequest of Frank B. Bristow, 1968), accession no. 2015.19.1530. Available through the National Gallery of Art's [Open Access](#) programme, under a CC0 licence.

Desk Used on the title page of Part II.

Ferdinand Cartier, *Desk* (c. 1940), graphite on paperboard. National Gallery of Art, Washington, Index of American Design, accession no. 1943.8.5338. Available through the National Gallery of Art's [Open Access](#) programme, under a CC0 licence.

Index

- abstract, 21
- area, 23, 24
- author, 12, 13
- background, 7
- bfseries, 15
- bg, 7
- cardbreak, 24
- centering, 17
- chapter, 14, 34
- chapter*, 14
- chaptermark, 12, 14
- colorlet, 10
- columns, 7
- cover, 12, 13, 37
- definecolor, 10
- documentclass, 10
- emph, 8, 16, 34
- eso-pic, 17
- font, 8
- footnote, 24
- Huge, 15
- item, 24
- justified, 8
- large, 34
- linewidth, 17, 20
- maketitle, 5, 6, 9, 12, 13, 34, 36, 37
- nameref, 23
- newfontfamily, 33
- noemph, 8
- nomultitoc, 8
- normalfont, 34
- oneside, 8
- openright, 8
- pagebreak, 24
- paragraph, 35
- part, 12-14, 28, 34, 36, 37
- part*, 12, 14
- ref, 23
- renewcommand, 10
- Rpg[X], 26
- Rpg[X]AddBoolean, 42, 43
- Rpg[X]AddProperty, 42, 43
- RpgAbstract, 21, 35
- RpgArea, 24
- RpgCard, 25, 26, 35, 38, 42, 44
 - Reset, 25
 - Set, 25
- RpgChapterName, 14
- RpgCMD, 10
- RpgContour, 9, 15
- RpgCoverTransparency, 13, 17
- RpgDefaultPartImage, 13
- RpgDice, 15, 38
- RpgDropCap, 10, 16, 35, 39
- RpgDropCapSeparation, 10, 16
- RpgFake, 15
- RpgFeat, 26
- RpgFiligree, 9, 20, 21
- RpgFiligreeSize, 10, 40
- RpgFont
 - AbstractBody, 21, 35
 - AbstractTitle, 21, 35
 - Body, 34
 - CardBody, 25, 35
 - CardHeader, 35
 - CardTitle, 35
 - Chapter, 34
 - DropCap, 16, 35
 - DropCapAfter, 16, 35
 - Emphasis, 16, 34
 - Footer, 35
 - Narration, 19, 35
 - PageNumber, 35, 36
 - Paragraph, 35
 - Part, 34, 37
 - SecretTitle, 22, 35
 - Section, 33, 34
 - SidebarBody, 19, 35
 - SidebarTitle, 19, 35
 - StatBlockBody, 35
 - StatBlockSection, 35
 - StatBlockTitle, 35
 - Subparagraph, 35
 - Subsection, 34
 - Subsubsection, 34
 - SubTitle, 13, 34
 - TableBody, 20, 35
 - TableHeader, 20, 35
 - TableTitle, 20, 35
 - TipBody, 20, 35
 - TipTitle, 20, 35
 - Title, 13, 34
 - TocChapter, 35
 - TocPart, 35
 - TocSection, 35
- RpgGetAuthor, 13, 34, 37
- RpgGetAuthor*, 13
- RpgGetKey, 26, 43
- RpgGetName, 14
- RpgGetSubtitle, 13, 34, 37
- RpgGetSubtitle*, 13
- RpgGetTitle, 13, 34, 37
- RpgGetTitle*, 13
- RpgIfEnv, 26
- RpgIfFont, 29
- RpgIfFontTF, 29
- RpgIfPaper, 29
- RpgIfPaperTF, 29
- RpgIfPreamble, 28, 30
- RpgIfPreambleTF, 28-30
- RpgIfSections, 30
- RpgIfSectionsTF, 30
- RpgIfSwitch, 18
- RpgIfTitle, 30
- RpgIfTitleTF, 30
- RpgItem, 26
- RpgKey, 8, 16, 34
- rpglatex, 47
- RpgMakeTitle, 12, 13
- RpgMap, 23, 39
 - Ref, 23, 24
 - Ref*, 23
- RpgNarration, 9, 19, 35, 38
- RpgNewSwitch, 18
- RpgOrdinal, 16, 17
- RpgPackagePath, 2, 9, 29, 47
- RpgPage, 16, 24
- RpgPart
 - Name, 13, 14
 - Transparency, 14
- RpgPlural, 17
- RpgPut, 17, 36
- RpgRowStyle, 20
- RpgSecret, 9, 22, 35, 38
- RpgSectionName, 14
- RpgSetTheme, 6, 8, 12, 28, 29
- RpgSetThemeColor, 9, 12
- RpgSetThemePath, 9, 12, 28
- RpgSidebar, 9, 19, 35, 38
- RpgSpell, 26
- RpgStat, 26
- RpgSubsectionName, 14
- RpgSubsubsectionName, 14
- RpgSuperOrdinal, 17
- RpgSwitch, 18, 22
- RpgTable, 9, 20, 35
- RpgTheme
 - [X]CardFormat, 42-44
 - [X]Format, 42, 43, 45
 - BoolKey, 31
 - CodeMapKey, 31
 - CreateFont, 33
 - DiceFormat, 38
 - DropCapHeight, 39
 - Font, 29, 33, 34
 - FooterDecoration, 7, 36
 - Keys, 31
 - MapFormat, 39
 - NewBoolKey, 30
 - NewCardableRuleEnv, 42
 - NewCodeMapKey, 30, 32
 - NewRuleEnv, 42, 43
 - NewTLChoiceKey, 30
 - NewTLKey, 30
 - NewTLMapKey, 30
 - OnPartChange, 37
 - PageNumberAt, 36
 - PaperColor, 7, 36
 - PaperImage, 7, 36
 - PartPage, 28, 37
 - SetFiligree, 40
 - SetSecretHidden, 40, 41
 - SetSecretVisible, 41
 - TitleCover, 37
 - TitleHeader, 37
 - TitleMode, 13, 37
 - TLChoiceKey, 31
 - TLKey, 31
 - TLMapKey, 31
- RpgTip, 9, 19, 21, 35, 38
- RpgToggle, 18
- RpgWholePageImage, 17
- section, 14, 33, 34
- section*, 14, 23
- sectionmark, 12, 14
- sections, 8
- setlength, 10
- size, 8
- subparagraph, 35
- subparagraph*, 23
- subsection, 14, 34, 44, 45
- subsection*, 14, 23
- subsectionmark, 12, 14
- subsubsection, 14, 34
- subsubsection*, 14
- subsubsectionmark, 12, 14
- subtitle, 13
- summary, 21
- tcbset, 38
- textbf, 16
- theme, 8
- themepath, 9

thepart, 37
title, 9, 12, 13

tl_if_blank:eTF, 38
tl_if_blank:nTF, 38

usepackage, 10